

AI 大模型不难

一本让零基础读者也能笑着看懂 Transformer 的开源电子书

共 29 课 · 五大部分 · 从零到面试通关

目录

开篇

第 0 课 · 开篇，这本书怎么读 + 大模型五分钟全景图

第一部分：世界观 —— 大模型到底是个什么东西

第 1 课 · Token、预训练、微调，三个词讲完大模型的一生

第 2 课 · 涌现和幻觉，大模型的两个「意料之外」

第 3 课 · 解码策略，模型怎么从概率里挑出下一个词

第 3 课 · 你怎么跟模型说话，Prompt + CoT + ICL

第 4 课 · RLHF 五分钟速览

第 5 课 · 省钱的三个办法，量化、蒸馏、分布式

第二部分：引擎 —— Transformer 里里外外

第 6 课 · Transformer 到底是什么

第 7 课 · Attention 公式，手写一遍就懂了

动手算一遍：句子「爱 学习」， $d_k = 2$

第 8 课 · 多头注意力，为什么一个头不够

第 9 课 · 位置编码，模型怎么知道词顺序

第 10 课 · 残差连接，梯度的高速公路

第 11 课 · LayerNorm，为什么每个 Transformer 层里都塞了一个它

第 12 课 · Pre-norm vs Post-norm

第 13 课 · 两种 Mask，为什么模型必须戴眼镜

第三部分：炼金 —— 模型是怎么训出来的

第 14 课 · 参数 vs 数据，容器和内容

第 15 课 · MLM vs CLM，两种截然不同的教法

第 16 课 · RLHF 全流程，SFT → RM → PPO

第 17 课 · DPO 和 GRPO，在 PPO 的基础上做了什么

第 18 课 · LoRA，穷玩大模型的终极答案

动手算一遍：一个 4×4 的权重矩阵， $r=2$

第 19 课 · 过拟合、梯度问题、Warmup、ZeRO

第四部分：落地 —— 大模型的十八般武艺

第 20 课 · RAG，让模型会查资料

第 21 课 · 推理优化全家桶

第 22 课 · AI Agent，让模型会干活

第 23 课 · MCP / A2A / Function Call，让模型会叫外援

第 24 课 · 前沿架构，MoE、GQA、DeepSeek 怎么玩的

第 24 课 · 多模态/VLM 入门，模型怎么看图

第五部分：实战 —— 面试现场还原

第 25 课 · 大厂真题精讲（上）

第 26 课 · 大厂真题精讲（下）+ 系统设计 + 项目讲述

第 0 课：开篇，这本书怎么读 + 大模型五分钟全景图

三个月前我准备 AI 技术面，背了一沓面试八股，背到第三天就崩溃了。不是因为记不住。是面试官每个词都能追着问，LoRA 原理？跟 QLoRA 的区别？为什么不用 Adapter？

后来我发现，大模型里那些让人头大的概念，每个都对应一个生活里的比喻。Attention 是图书馆查资料。残差连接是传话游戏的复印件。LayerNorm 是把全班身高统一成标准尺。

这本书就是把这些比喻整理了出来。26 课，分五部分。

第一部分，世界观（1-5 课）：Token、预训练、微调，涌现和幻觉，Prompt 工程，RLHF，量化蒸馏分布式。

第二部分，引擎（6-13 课）：Transformer 架构，Attention 公式，多头注意力，位置编码，残差，归一化，Pre/Post-norm，两种 Mask。面试里被问得最多的八课。

第三部分，炼金（14-19 课）：参数 vs 数据，MLM vs CLM，RLHF 全流程，DPO/GRPO，LoRA，训练硬核。

第四部分，落地（20-24 课）：RAG，推理优化，Agent，MCP 协议，前沿架构。

第五部分，实战（25-26 课）：十道大厂真题 + 系统设计。

零基础顺着读。面试突击跳到 25-26 课刷真题。有基础的跳着当词典查。

AI 大模型世界地图

26 课，五站旅程——从零到面试通关

开篇

第 0 课 · 地图

第一部分：世界观 —— 大模型到底是个什么东西

第 1-5 课：Token · 涌现与幻觉 · Prompt/CoT/ICL · RLHF 速览 · 量化/蒸馏/分布式

第二部分：引擎 —— Transformer 里里外外

第 6-13 课：Transformer · Attention 手写 · 多头注意力 · 位置编码 · 残差连接 · LayerNorm · Pre/Post-norm · 两种 Mask

第三部分：炼金 —— 模型是怎么训出来的

第 14-19 课：参数量/数据量 · MLM/CLM · RLHF 全流程 · DPO/GRPO · LoRA · 训练硬核

第四部分：落地 —— 大模型的十八般武艺

第 20-24 课：RAG · 推理优化 · Agent · MCP/A2A · 前沿架构(MoE/DeepSeek)

实战

第 25-26 课 · 大厂面试

AI 大模型世界地图

磨平一些信息差。

第 1 课：Token、预训练、微调，三个词讲完大模型的一生

Token — 模型的字母表

模型不直接读汉字或英文单词。它读的是 Token，一种介于「字符」和「词」之间的最小处理单元。

Tokenization 的主流算法是 BPE (Byte Pair Encoding)。它从字符起步，反复合并最常相邻出现的字符对，逐步构建词表。「我喜欢」可能先合并「喜」+「欢」→「喜欢」，再合并「我」+「喜欢」→「我喜欢」。高频词合并成大块，生僻字保持碎片。GPT-4 的词表约 10 万个 Token，全世界的语言都在这 10 万块积木里拼。

一个 Token 约等于 0.75 个英文单词，或 0.5 个汉字。「我要一杯珍珠奶茶」约 6 个 Token，高频词组「珍珠奶茶」作为一个整体占 1 个 Token。

Token 进了模型之后，被映射成一个高维向量 (Embedding)，比如 768 维。这个向量不是人为定义的，是训练过程中自己学会的。意义相近的词向量距离近，「猫」和「狗」的向量夹角比「猫」和「汽车」小得多。

预训练 — 猜了几十万亿次的下一个词

给了模型几十万亿 Token 的互联网文本。训练方法极其简单：让它猜下一个词。

「今天天气怎么」→ 模型预测下一个 Token 是「样」。猜对了，参数不动。猜错了，算 Loss (交叉熵损失，衡量预测分布和正确答案之间的差距)，反向传播，梯度下降更新参数，再去猜下一个。

每猜错一次就微调一次参数。几十万亿次之后，模型从随机噪声变成了一个能组织语言的大脑。它学会了语法 (形容词修饰名词)、常识 (下雨要带伞)、甚至一些推理 (如果 $A > B$ 且 $B > C$ ，那么 $A > C$)。没有人教它语法规则，它从几十万亿次「猜词-纠错」的循环里自己归纳出来的。

这几十万亿次计算的总成本：GPT-4 级别约 1 亿美元的电费和 GPU 租金。这也是为什么只有少数公司能训基座模型。

微调 — 通用大脑做特定训练

预训练完的模型什么都会一点，但什么都不精。你问它客服问题，它可能回答得像散文。微调就是在特定领域的高质量数据上再做几轮猜词训练。

准备 5000 条奶茶店客服对话：「珍珠奶茶有中杯和大杯吗」「有，中杯 15 元大杯 20 元」，让模型在这些数据上继续猜下一个词。参数从预训练的「通用语言」往「奶茶店客服」方向微调。

底层的语言能力还在（语法、常识），但输出风格、知识侧重、行为模式全调整了。LoRA 微调只改不到 1% 的参数就能做到，第 18 课详讲。

走一遍完整流程：奶茶店客服机器人

Step 1, Tokenize: 「我要一杯珍珠奶茶少糖」→ BPE 切成 [我要, 一杯, 珍珠, 奶茶, 少, 糖]。6 个 Token。

Step 2, 预训练模型处理: 每个 Token 映射成 768 维 Embedding → 过 32 层 Transformer → 输出下一个 Token 的概率分布。它知道「珍珠奶茶」是饮品、「少糖」是甜度选项。但它不知道你们店的菜单。概率前三是「好的请稍等」「加珍珠吗」「建议搭配炸鸡」。

Step 3, 微调后: 反复在奶茶店真实对话上训了几千步。模型不再提炸鸡，它学会了推销第二杯半价、确认杯型和温度、告知取餐时间。只改了不到 1% 的参数，行为全变了。

Token 是原料，预训练是上学，微调是实习。

磨平一些信息差。

第 2 课：涌现和幻觉，大模型的两个「意料之外」

GPT-3 在 130 亿参数时不会加法，650 亿也不会，1300 亿也不会。到了 1750 亿，突然会了。不是从 60 分涨到 80 分，是从 20 分跳到 80 分。

涌现：为什么参数堆到某个量级，能力会跳变

这跟 Scaling Law 直接相关。第 14 课会讲公式，这里先讲直觉。

Loss 随参数量 N 和数据量 D 的增长是平滑的幂律下降： $L \propto 1/N^\alpha + 1/D^\beta$ 。但 Loss 是一个全局指标，它衡量模型在所有 token 上的平均预测准确率。具体到某个任务（比如三位数加法），Loss 从 4.0 降到 3.5 的阶段可能准确率一直停在 20%。但当 Loss 降到某个临界点（比如 3.2 以下），模型突然「学会」了加法，准确率跳到 80%。

Loss 是平滑的，但能力是突变的。因为任务本身有一个「最小所需精度」阈值。就像打靶：靶子 10 环，你从脱靶练到 3 环再练到 6 环都是不及格（没命中靶心）。但一旦突破到 7 环以上，开始稳定命中，在外人看来就是「突然会了」。

涌现不是魔法，是平滑优化曲线在离散评测指标上的相位突变。和物理里的相变完全一个道理：水从 99°C 到 100°C 的温度变化是线性的，但物态从液态到气态是跳跃的。

幻觉：猜词机器为什么会「编造」

模型就是一个猜下一个词的概率机。给定 prompt「推荐三本明朝历史的书」，它开始逐 token 生成。生成第一个词时，它在 10 万 Token 词表上输出概率分布。输出「《」的概率最高，输出「万」的概率也差不多，输出「明」的概率排第三。

注意：它不是在「检索知识」。它是在计算给定上下文后，哪个 Token 出现的条件概率最高。它看到的训练数据里有无数「书名 = 《XXXX》」的模式。所以它顺着这个模式往下猜，《大明》《明朝》《明史》...逐步拼出一串看起来像书名的 Token。

当它不认识任何真正的明朝历史书时，它不会停。条件概率仍然存在，「万历十五年」出现在「明朝」「历史」「推荐」这些上下文里的概率比「香蕉飞船」高得多。所以它输出了一本不存在的书，但书名的每个字在统计意义上都是「合理的」。

RLHF 让问题更微妙。RM 训出来之后，模型学到一个规律：「给出确定答案」的回答得分比「我不知道」高。于是面对不确定性，模型倾向于猜一个最高概率的 Token 序列而不是承认无知。

正确防御：给 prompt 加「如果信息不确定，请直接说明不知道，不要猜测」。这句话改变了条件概率分布，「我不知道」这个 Token 序列的概率被抬高，模型更容易选择承认无知。

走一遍幻觉是怎么产生的

你问：「《大秦帝国》孙皓晖写的，总共有几卷？」

模型训练数据里见过「大秦帝国」「孙皓晖」「历史小说」共现，但没记住确切卷数。它不输出「我不知道」，因为 RLHF 训出的行为是「尽量给答案」。

它开始猜词：历史小说通常是三到六卷，大秦帝国这种大制作更可能五到六卷。逐 Token 生成「六」，然后继续生成「卷」，然后为显得专业，再加「分别是」然后编出了六卷的名字。每一步的 Token 在统计上都是合理的，只是它们的组合对应了一本不存在的书。

磨平一些信息差。

第 3.5 课：解码策略，模型怎么从概率里挑出下一个词

Attention 算完了，Transformer 跑完了，Softmax 输出了一个概率分布：[0.02, 0.15, 0.45, 0.08, 0.30]。总共 10 万个候选 Token，每个有个概率。现在要挑一个出来当下一个词。

怎么挑？五种挑法。

Greedy Search — 永远选概率最大的

每步挑概率最高的 Token。快，确定性强，每次同样的 prompt 永远输出同样的结果。致命伤：单调重复。因为一旦第一步选了「喜欢」，下一步「喜欢」之后最高概率的又是「你」，然后「喜欢」「你」「喜欢」「你」无限循环。不做任何创意生成。

面试时说：Greedy 适合翻译、代码补全这类确定性任务。不适合开放式写作和对话。

Beam Search — 保留K条候选路

不只保留最优的一条路，而是保留 K 条。K=4 就是 4 条。每一步对每条路径的所有可能 Token 做扩展，算累积概率，砍掉不行的，保留 Top-K。

质量比 Greedy 高得多，「我昨天去了动物园，看到了一头大象」的概率比「我喜欢你喜欢你」高几百倍，Beam Search 能找出来。代价是 K 倍计算量，K 越大越贵。

面试时说：Beam Search 是翻译和摘要的标配。但 K 太大 (>10) 边际收益递减，K=3-5 最实用。

Top-K Sampling — 从Top-K里随机抽

不选固定的，从概率最高的 K 个 Token 里随机抽一个。K=50 就是从 Top-50 里按概率权重随机选。

引入随机性解决了重复循环。但如果 Top-50 里有一堆低概率的奇怪 Token 混进来，可能偶尔输出离谱词。

面试时说：K 是静态的，不随分布变化。一个很集中的分布（第一个词 0.9，其余 0.1），K=50 会把大量低概率 Token 纳入候选。一个很分散的分布（所有词都差不多），K=50 可

能砍掉合理候选。

Top-P (Nucleus) Sampling — 累积概率超过P就停

自适应版 Top-K。从概率最高的 Token 开始累加，累到超过 P（通常 0.9）就停，只在这个最小集合里随机抽。

分布集中时，「the」概率 0.85，「a」0.08，Top-P 只需要 2 个 Token 就超过 0.9。分布分散时，20 个 Token 各有 0.04-0.06 概率，Top-P 会纳入所有 20 个。

GPT 系列的默认策略就是 Top-P，P 通常 0.9-0.95。

面试时说：Top-P 比 Top-K 更鲁棒，因为它随分布自动调整候选集大小。实际工程中两者常混用：先 Top-K 砍到 50，再 Top-P 过滤。

Temperature — 控制概率分布的平滑度

在 Softmax 之前，把 logits 除以温度 T：

```
softmax(logits / T)
```

T=1：原始分布，不改。T=0.5：分布更尖锐，高概率更高、低概率更低，Greedy 和 Sampling 都更确定。T=2.0：分布更平滑，各 Token 概率差距缩小，Sampling 更随机多样。

T→0 逼近 Greedy。T→∞ 变成完全随机采样。标准区间：T=0.7-1.0 适合对话，T=0.2-0.5 适合代码/翻译，T>1.0 适合创意写作。

面试时说：Temperature 是控制「创造性 vs 确定性」的最直观旋钮。面试官问你调过什么参数来改善输出质量，Temperature 是最安全、最管用的答案。

一句话选型

任务	策略	温度
翻译、代码	Beam Search (K=3-5)	0.2-0.5
对话、客服	Top-P (0.9)	0.7-0.8

任务	策略	温度
创意写作、头脑风暴	Top-P (0.95) + Top-K (50)	1.0-1.2
数学推理	Greedy + CoT	0.0-0.1

磨平一些信息差。

第 3 课：你怎么跟模型说话，Prompt + CoT + ICL

Prompt Engineering 的核心就一件事：把要求写具体。「写个方案」不行，「写一个针对大学生的校园奶茶店营销方案，预算 5000 元，分线上线下各三项措施」才行。

记忆框架 CRISP。Context 上下文，你是谁什么场景。Role 角色，你是一个毒舌美食评论家。Instruction 指令，写 300 字分三段。Style 风格，用初中生能看懂的语言。Purpose 目的，这份材料要给投资人看的。五要素配齐，输出质量直接上一个台阶。

CoT — 为什么「一步步思考」真的有用

不是魔法。是自回归生成的自然结果。

模型一次只生成一个 Token。没有 CoT 时，它面对「小明分糖果还剩几颗」这个问题，必须在第一个输出 Token 就给出最终答案。这对一个概率模型来说，是把多步推理压缩到一步，中间每一步都可能算错。

有了 CoT，提示「我们一步步来」。模型的第一个输出变成「步骤 1：小明最初有 15 颗糖」。这个输出立即被追加到对话历史里，作为下一次生成的上下文。所以生成「步骤 2」的时候，模型不仅能看到原始题目，还能看到自己刚生成的「步骤 1」的结果。

CoT 的本质是让模型的输出变成自己的输入。它自己给了自己额外的推理锚点。每一步的正确结果通过自回归机制自然传导到下一步，不是模型变聪明了，是它的工作记忆变大了。

这和人类做复杂数学题要打草稿是同一个道理。不是你的大脑在草稿纸上变聪明了，是你把中间结果写了下来，下一步不用重新算。

ICL — 不训练就学会新任务

In-Context Learning。在 prompt 里贴 2-3 个例子，模型看完就能推断任务模式，零参数更新。

ICL 的机制：贴进去的例子经过 Transformer 的前向传播后，它们的 KV 向量留在了注意力层里。当模型处理真正的 query 时，这些「例子」的 KV 向量可以通过自注意力被查询，

模型从例子中提取任务模式，直接应用到当前 query 上。

不是「学习」。是「用注意力在上下文中找到了最相似的模板然后跟着做」。参数一个都没动。整个过程就是一次前向传播。

例子质量和多样性决定效果。贴三个例子都是一种模式，模型对第四种模式就懵。贴三个不同模式但每个都清晰标注，模型能同时学会四种。

ICL 是怎么生效的，一个情感分类走一遍

假设你要模型判断一段评论的情感（正面/负面），但不想微调。

不用 ICL，直接问：「这家店的奶茶太甜了，等杯等了我二十分钟。判断情感。」模型可能猜对，也可能猜错。它从 prompt 里拿不到任何关于「判断情感」这个任务格式的信号。

用 ICL，贴三个例子在前：

评论：这家店的服务态度超好，下次还会再来。情感：正面
评论：奶茶是温的，珍珠硬得嚼不动。情感：负面
评论：环境不错，适合朋友聊天，就是停车不太方便。情感：中性

评论：这家店的奶茶太甜了，等杯等了我二十分钟。情感：

模型读完前三个例子，注意力机制已经捕捉到了模式，「评论 + 情感标签」的输入输出格式，以及「服务好→正面」「珍珠硬→负面」「有优点也有缺点→中性」的分类逻辑。它不用训练，跟着这个模式输出「负面」。

ICL 的机制拆开。三个例子的 Token 经过 32 层 Transformer 后，它们的 Key 和 Value 向量留在了每一层的注意力缓存里。当模型要预测「情感：」后面的词时，Query 向量会在所有层的 KV 缓存里搜索跟当前任务格式最相似的模板。「态度超好→正面」这个先例的 KV 被你当前的问题 Query 命中了，模型照着它的答案格式继续往下走。

参数一个没动。前向传播结束就把这些 KV 缓存扔掉了。ICL 就是一次手把手的现场示范，不是一次微调。

CoT 是怎么生效的，一个数学题走一遍

题目：「小明有 15 颗糖，给了小红 3 颗，又买了 5 颗，然后分给 4 个同学每人一样多，还剩几颗？」

不用 CoT，模型直接从 prompt 生成答案，它得在一个输出里算出结果。经常错。

用 CoT，加一句「我们一步步算」：

步骤 1：小明最初有 15 颗糖
步骤 2：给小红 3 颗， $15 - 3 = 12$ 颗
步骤 3：又买 5 颗， $12 + 5 = 17$ 颗
步骤 4：分给 4 个同学， $17 \div 4 = 4$ 剩 1
答案：1 颗

模型生成步骤 1 时，只看原始题目。生成步骤 2 时，看原始题目 + 步骤 1 作为上下文。生成步骤 4 时，上下文里已经有了前 3 步的正确结果。CoT 把一个大推理拆成了 4 个小推理链，每一步的正确答案通过自回归自然流入下一步。

磨平一些信息差。

第 4 课：RLHF 五分钟速览

GPT-3 会说人话但不知道什么该说什么不该说。问它讲故事它会，问它做炸弹它也给步骤。不是邪恶，是没有「好坏」概念。

RLHF 三步，像训练脱口秀演员。

第一步 SFT，请家教。人类标注员手写几千条「完美回答」，模型照着学。学完能从「会说完整句子」变成「能写及格段子」。

第二步 RM，请评委。标注员给同一个问题的多个回答排序，A 比 B 好，C 比 D 差。训练一个打分模型，学会判断回答的质量。

第三步 PPO，反复彩排。模型生成回答，RM 打分，根据分数调策略。高分多保留，低分少用。一万遍后模型能稳定输出高质量回答。

用一句话走通整个 RLHF 流程

Prompt：「推荐一本入门 Python 的书」

原始预训练模型可能答什么：从 Reddit 学到「买我的课」「扫码加群」这种垃圾回复。因为它见过的网页里这种文案太多了。

SFT 阶段：人类标注员写了 50 条「理想回答」范例，推荐《Python 编程从入门到实践》，说明适合零基础、有练习题、第三版更新到 Python 3.11。SFT 训练后，模型学会了回答应该「具体、有用、不卖课」。

RM 阶段：给 SFT 模型同一个 Prompt，让它生成 4 条回答。标注员排序：

- A（五星）：推荐一本经典入门书 + 讲清楚为什么适合 + 附学习路径
- B（四星）：推荐一本书 + 简单介绍
- C（两星）：列了五本书但没说明哪本好
- D（零星）：「买我的课，7 天学会 Python」

RM 学会给 A 打高分、D 打零分。

PPO 阶段：模型生成回答 → RM 打分 → 反向传播调参数。生成 D 类的回答概率被压低，生成 A 类的被抬高。一万轮后模型不再输出卖课文案。

磨平一些信息差。

第 5 课：省钱的三个办法，量化、蒸馏、分布式

量化 — 把每个参数的「精确值」换成「近似值」

大模型是一堆数字。7B 模型就是 70 亿个数字。每个数字默认用 32 位二进制存储 (FP32)，70 亿个 32 位数字 = 28 GB。

但模型真的需要这么精确吗？训练时确实需要，梯度更新要精确到小数点后六位。推理时不需要，参数只做一次前向计算，多一点误差对结果影响极小。

量化的操作很简单：把每个 32 位的数值，用更少的位数去近似。就像你用「127」这个刻度尺量东西和用「7」这个刻度尺量东西。127 格能标出 0.720 和 0.718 的区别，7 格只能分出大概在第 3 格还是第 4 格。但在 70 亿个参数里，揪着一个小数点后三位的精度不放，对「猫坐在垫子上」这个语义没影响。模型学到的是大尺度的结构，「猫」和「坐」的语义关系、「垫子」和「上」的位置关系，不是某个注意力权重到底是 0.718382 还是 0.72。

FP16 用 16 位，精度轻微下降但肉眼不可见。INT8 用 8 位，精度再降一档，但 0.72047 和 0.718382 的区别在全连接层里被稀释了。INT4 用 4 位，差距开始明显，但 QLoRA 在量化后的 INT4 模型上再加 LoRA 训练，能把这个精度损失补回来。

显存变化：FP32 → INT8，28 GB 变 7 GB（砍四分之三）。INT4 再砍一半到 3.5 GB。一张 4090 显卡就能跑 7B 模型。

蒸馏 — 大模型教小模型，不是抄答案，是学思路

蒸馏不是让小模型直接背大模型的输出。它利用大模型输出的「软标签」，不是「答案是 B」这种硬标签，而是「A 的概率 2%，B 的概率 85%，C 的概率 10%，D 的概率 3%」这种完整概率分布。

这些概率分布里包含了大模型的「知识」：虽然正确答案是 B，但 C 也有 10% 的概率，说明 C 和 B 有一定相关性。小模型学习这个分布，相比只学「答案是 B」，能获得更丰富的训练信号。

蒸馏的损失函数：

$$\text{Loss} = \alpha \cdot \text{KL}(\text{p_soft} \parallel \text{q}) + (1-\alpha) \cdot \text{CrossEntropy}(\text{y_true}, \text{q})$$

两个来源的信号混合。KL 散度那项让小模型的输出分布逼近大模型的软标签，CrossEntropy 那项确保不能偏离真实答案。 α 是混合系数，通常 0.7-0.9。温度 T 是另一个关键参数，软标签在送入 KL 前先除以 T （通常 2-5），让分布更平滑。温度越高，大模型给的「C 也有 10% 概率」这类副信息越明显，小模型学到的细节越多。

蒸馏出来的 7B 小模型能达到 70B 大模型 80%-90% 的水平，但推理成本只有几十分之一。手机上跑的那些 AI 助手、输入法预测，全是蒸馏出来的。

分布式训练 — 一张 GPU 不够，多张怎么分工

大模型太大，一张 GPU 显存放不下。解决方案分两种思路。

思路一，数据并行。4 张 GPU，每张存一份完整的模型。把训练数据切成 4 份，每人算自己那份的梯度。算完之后，4 张卡需要互相通信，把各自的梯度汇总成一个平均值。这个通信操作叫 all-reduce，每张卡把自己的梯度发给其他 3 张卡，同时接收别人的，最后 4 张卡手里都拿到完全相同的平均梯度。各自用这个平均值更新参数，4 张卡的参数始终保持一致。加速了训练时间（4 张卡同时算），但不省显存（每张卡都要存完整的模型）。

问题：4 张卡通信快，128 张卡互相传梯度的耗时可能比算梯度本身还长。缓解办法叫梯度累积，不要每算一个小 batch 就通信一次，攒够 N 个小 batch 的梯度累加起来再通信。省了通信频次但训练会慢一些，因为攒梯度的过程中 GPU 闲着等。

思路二，模型并行。模型太大了，一张卡连模型本身都放不下。那就把模型切成两半：前半段放 GPU 0，后半段放 GPU 1。数据先过 GPU 0 算完前半段，结果传给 GPU 1 续算。GPU 0 算的时候 GPU 1 在等，GPU 1 算的时候 GPU 0 在等，总有一张卡闲着。这浪费叫流水线气泡（pipeline bubble）。好处是显存省一半，每张卡只存半个模型。代价是 GPU 利用率掉了一截。

实际千卡训练三种方案混合：8 台机器之间做数据并行，每台机器内部 8 张卡做模型并行。再叠上第 19 课讲的 ZeRO，把每张卡上的优化器状态也切碎分给其他卡存，能进一步省显存。1.6T 参数的 DeepSeek-V3 就是这么搭出来的。

量化到底损失了多少精度，一个参数走一遍

拿一个注意力层的权重矩阵，里面有个值是 0.718382 (FP32, 32 位浮点数)。

FP16 量化: $0.718382 \rightarrow 0.71875$ 。误差 0.0004，肉眼不可见。

INT8 量化: 0.718382 先缩放到 $[-127, 127]$ 范围。这一层最大值 1.5，最小值 -1.5， $(0.718382 / 1.5) \times 127 \approx 60.8 \approx 61$ 。解码回来 $61 / 127 \times 1.5 \approx 0.72047$ 。误差 0.002，基本不影响结果。

INT4 量化: 范围只有 $[-7, 7]$ 。 $(0.718382 / 1.5) \times 7 \approx 3.35 \approx 3$ 。解码回来 $3 / 7 \times 1.5 \approx 0.6429$ 。误差 0.075，数学推理崩了。但 QLoRA 在量化后的模型上再加 LoRA 微调，训练过程中能把这个误差补回来。

量化是减肥，蒸馏是拜师，分布式是喊人搬砖。三种方法经常一起上：先蒸馏成小模型，再 INT4 量化，分布式部署到手机端。

磨平一些信息差。

第 6 课：Transformer 到底是什么

2017 年 Google 发了篇论文，《Attention Is All You Need》。没吹牛。GPT、Claude、Llama、DeepSeek 全是它的直系后代。

要理解 Transformer 为什么是革命，得先知道它革了谁的命。

RNN：一条路走到黑

在 Transformer 之前，处理文本的主流方案是 RNN（循环神经网络）。

RNN 的核心操作极其简单：它一次只读一个词，读完更新自己的「记忆」，再读下一个。数学上就是一个循环公式：

$$h_t = f(W \cdot h_{t-1} + U \cdot x_t)$$

h_t 是读完第 t 个词之后的记忆状态。 h_{t-1} 是前一步的记忆。 x_t 是当前词。 W 和 U 是可训练的权重矩阵。 f 是激活函数。每读一个词，就用当前词 + 上一步记忆，算出新的记忆。

读「猫坐在垫子上」的过程：

t=1: 读「猫」	$\rightarrow h_1 = f(W \cdot h_0 + U \cdot \text{「猫」})$	记忆里只有「猫」
t=2: 读「坐」	$\rightarrow h_2 = f(W \cdot h_1 + U \cdot \text{「坐」})$	记忆里有「猫」和「坐」
t=3: 读「在」	$\rightarrow h_3 = f(W \cdot h_2 + U \cdot \text{「在」})$	逐步积累
t=4: 读「垫子」	$\rightarrow h_4 = f(W \cdot h_3 + U \cdot \text{「垫子」})$	
t=5: 读「上」	$\rightarrow h_5 = f(W \cdot h_4 + U \cdot \text{「上」})$	最终状态包含整句话

两个致命问题。第一，串行。 h_5 必须等 h_4 算完， h_4 等 h_3 。5 个词要算 5 步，GPU 大部分时间在等，不能并行。第二，长距离遗忘。 h_t 是通过 W 反复相乘传下来的。 W 的值通常略小于 1，连乘 100 次之后，100 步前的信息几乎消失。「猫」这个词的信息传到「垫子」时已经稀释了几百倍。

LSTM 和 GRU 加了「门控」机制缓解遗忘，核心是一套「记住什么、忘掉什么」的选择开关。长距离记住了，但串行的本质没变。

Transformer：所有词同时互相看

Transformer 的核心操作不再是按顺序读。它的自注意力机制一次性把所有词扔进一个计算图，让每个词同时「查询」所有其他词。

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$$

不需要等上一步。不需要通过 W 反复相乘传递信息。每个词通过 QK^T 矩阵乘法直接跟所有其他词算相关性。第 1 个词和最后 1 个词之间的「距离」不是 100 步递推，而是一次点积。长度 5 还是 50000，一次矩阵乘法全解决。这在第 7 课会拆开讲。

两个字面叫「自注意力」，Q、K、V 都来自同一份输入数据，是「自己看自己」。

Encoder — 一眼看完全文

Encoder 的任务是把输入句子变成一串「理解过的」向量。输入 5 个 Token，输出 5 个等长的向量，每个向量里融合了整句话的上下文信息。

它用双向自注意力：处理「猫」这个位置时，不光看前面的「猫」，也看后面的「垫子」「上」。「猫」的最终向量是整句话所有词的加权混合。

为什么需要双向？因为理解「猫坐在垫子上」这句话里的「猫」，上下文信息都重要。「猫」是主语（后面的「坐」告诉你的），「猫」是什么（前面的定冠词或修饰语告诉你的）。只看一边，理解是不完整的。

BERT 只用了 Encoder。因为分类、命名实体识别、问答理解这些任务不需要生成新文本，只需要把输入「读懂」。

Decoder — 一个词一个词往外蹦

Decoder 的任务是生成。输入 5 个 Token（或是 Encoder 的输出），逐个位置预测下一个词。

它的自注意力是因果的，加了下三角 Mask。生成第 3 个词时只能看到第 1、第 2 个词，不能看第 4、第 5。因为推理时第 4 个词还不存在。

GPT 只用了 Decoder。去掉 Encoder，去掉 Cross-Attention，只保留因果自注意力。输入 prompt 「今天天气」，逐 Token 预测下一个词，把生成的词再喂回去当输入，这就是自回归生成。

只用 Decoder 能「理解」prompt 吗？能。因果自注意力虽然不能看未来，但能看全部过去。你的 prompt 就是它的过去。读到「猫坐在」这三个字要生成下一个词时，「猫」「坐」「在」都在它的注意力范围里。

Cross-Attention — Decoder 向 Encoder 要信息

Encoder 已经「读懂了」输入。Decoder 在生成时除了看自己已生成的词，还要查 Encoder 的输出。这个机制叫 Cross-Attention。

Cross-Attention 和自注意力的公式完全一样，区别是 Q、K、V 的来源不同：

	Q（我要查什么）	K、V（被查的资料）
自注意力	来自同一份输入	来自同一份输入
Cross-Attention	来自 Decoder 当前位置	来自 Encoder 的全部输出

Decoder 在生成「like」这个词时，用自己的 Query 去查 Encoder 输出的 Key，发现「喜欢」这个位置最相关，就从 Encoder 的 Value 中提取「喜欢」的语义。翻译场景最直观：Encoder 读中文，Decoder 写英文，Cross-Attention 就是查字典。

T5 是 Encoder-Decoder 完整版。机器翻译、文本摘要这种输入和输出不是同一种「语言」的任务，用完整版最自然。

三种变体一句话

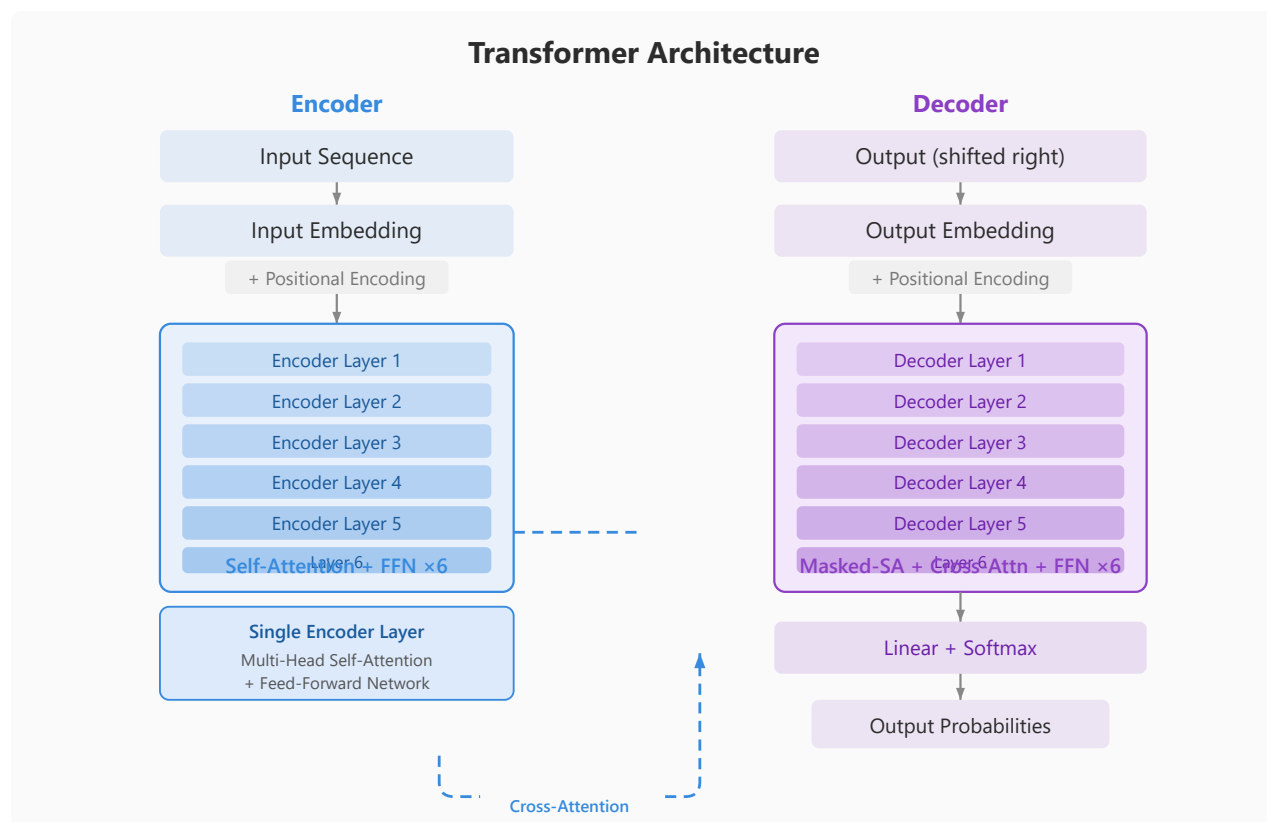
模型	用了什么	适合什么
BERT	只有 Encoder	理解：分类、NER、问答
GPT	只有 Decoder	生成：对话、写作、代码
T5	Encoder + Decoder	转换：翻译、摘要

共享的砖块只有四种

不管怎么组合，Encoder 还是 Decoder，每层里面反反复复就用这四种零件：

注意力机制 — 每个词跟所有其他词对话。残差连接 — 梯度高速公路，防深层训不动。层归一化 — 统一数值范围，防爆炸和消失。位置编码 — 注入词序，不然模型分不清「猫追老鼠」和「老鼠追猫」。

GPT-4 堆了几百层，每层的砖还是这四种。堆得多，能力就大。就这么简单。



Transformer 架构图解

走一遍：一句话穿过 GPT 的完整路径

输入：「猫坐在垫子上」

Step 1 — Tokenize + Embedding：切成 5 个 Token，每个映射成 768 维向量。

Step 2 — 位置编码：位置 0 的「猫」旋转一个角度，位置 1 的「坐」转更大角度。现在每个向量知道自己排第几。

Step 3 — Attention 层（第 1 层）：「垫子」通过 Attention 发现自己跟「猫」「坐」关系最紧，跟「上」的关系没那么紧。所有 Token 互相看，加权融合信息。

Step 4 — 残差 + LayerNorm：加回原始输入，归一化。

Step 5 — FFN 层：每个 Token 独立过两层全连接，做一次非线性变换。

Step 6 — 重复 32 层：每层都在上一步基础上提取更高级的语义关系。第 1 层可能学到「垫子」跟「猫」相关，第 20 层可能学到整句话的动作逻辑。

Step 7 — 输出：最后一个 Token 的向量过 Linear + Softmax，从 10 万 Token 词表里挑概率最高的那个，输出下一个词。

磨平一些信息差。

第 7 课：Attention 公式，手写一遍就懂了

Attention 像在图书馆查资料。Q 是你的问题，K 是每本书的标签，V 是书的内容。

公式四步。

第一步， $Q \times K^T$ 。计算你的问题和每本书标签的相似度。结果是 $n \times n$ 矩阵，每个 token 对每个 token 的相关性分数。这就是 $O(n^2)$ 的来源。

第二步，除以 $\sqrt{d_k}$ 。Q 和 K 的维度大了点积值会爆炸，送进 softmax 后梯度消失。除以 $\sqrt{d_k}$ 把数值压回稳定区间。不除的话训练会发散。

第三步，softmax。把分数变成概率，每一行加起来等于 1。 e^x 让高分更突出、低分接近零。但 $e^0 = 1$ ，所以即使两个词完全正交，注意力也不会是零，每个词都能分到一点关注。

第四步， $\times V$ 。按 softmax 算出的权重对 V 做加权求和。「猫坐在垫子上」里，「坐」的输出 = $V_{\text{坐}} \times 0.5 + V_{\text{猫}} \times 0.3 + V_{\text{垫子}} \times 0.15 + V_{\text{上}} \times 0.05$ 。它不是「坐」的原始含义，而是融入了上下文的新表示。

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$

就这一行。四行 PyTorch 写完。

```
def attention(Q, K, V):
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / (d_k ** 0.5)
    return F.softmax(scores, dim=-1) @ V
```

动手算一遍：句子「爱学习」， $d_k = 2$

设 Q、K、V 如下（故意设得极简，方便手算）：

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad V = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

爱 学习 爱 学习 爱 学习

Step 1: $Q \times K^T$

$$\begin{aligned} [1,0] \cdot [1,0]^T &= 1 & [1,0] \cdot [0,1]^T &= 0 \\ [0,1] \cdot [1,0]^T &= 0 & [0,1] \cdot [0,1]^T &= 1 \end{aligned}$$

$$QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

「爱」和「学习」正交，交叉为 0。只有自己跟自己相关。

Step 2: $/ \sqrt{2} \approx / 1.414$

$$\begin{bmatrix} 0.71 & 0 \\ 0 & 0.71 \end{bmatrix}$$

Step 3: Softmax (按行)

第一行「爱」: $e^{0.71}=2.03, e^0=1, \text{sum}=3.03 \rightarrow [2.03/3.03, 1/3.03] = [0.67, 0.33]$

第二行「学习」: $e^0=1, e^{0.71}=2.03 \rightarrow [1/3.03, 2.03/3.03] = [0.33, 0.67]$

$$\text{Attention Weights} = \begin{bmatrix} 0.67 & 0.33 \\ 0.33 & 0.67 \end{bmatrix}$$

注意：原始交叉为 0，但 $e^0=1$ ，所以不是零。注意力有保底。

Step 4: $\times V$

$$\begin{aligned}\text{Output}_{\text{爱}} &= 0.67 \times [2, 0] + 0.33 \times [0, 2] = [1.34, 0.66] \\ \text{Output}_{\text{学习}} &= 0.33 \times [2, 0] + 0.67 \times [0, 2] = [0.66, 1.34]\end{aligned}$$

「爱」的输出不只是自己的 $[2, 0]$ ，混入了 33% 的「学习」。「学习」的输出不只自己的 $[0, 2]$ ，混入了 33% 的「爱」。

这就是 Attention 的核心：每个 token 的输出是所有 token 的加权混合。正交不代表绝缘，softmax 的保底机制让信息永远在流动。

磨平一些信息差。

第 8 课：多头注意力，为什么一个头不够

上节课讲了 Attention 的核心公式，Q 和 K 点积得到一个标量，表示两个词的相关性。但语言里同时存在几十种关系，主谓、动宾、指代、修饰、时间、地点，如果只靠一个标量来描述「猫」和「坐」之间的所有关系，结果只能是模糊的平均值。

多头的数学机制

单头注意力：输入 X 的维度是 512，通过三个矩阵 W_Q 、 W_K 、 W_V （都是 512×512 ）各投影一次，得到 Q、K、V 都是 512 维。然后做 QK^T ，得到 $n \times n$ 的注意力矩阵。

多头注意力：不做 512 维的全量投影。而是把 Q、K、V 各自切成 8 份，每份 64 维。

$$Q = [Q_1, Q_2, \dots, Q_8], \quad K = [K_1, K_2, \dots, K_8], \quad V = [V_1, V_2, \dots, V_8]$$

每个头在自己的 64 维子空间里独立运行一套完整的 Attention：

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax}(Q_i K_i^T / \sqrt{64}) V_i$$

8 个头的输出拼接起来：

$$\text{MultiHead} = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_8) \cdot W_O$$

Concat 的结果是 $8 \times 64 = 512$ 维，和单头一样大。最后用一个 512×512 的 W_O 矩阵融合 8 个视角。

为什么 64 维够用，512 维反而不好

注意力分数 $Q_i \cdot K_j$ 是一个标量。不管你用 64 维还是 512 维，两个词之间最终只输出一个数字。1 个头 = 1 个数字描述所有关系，8 个头 = 8 个数字各描述一种关系。

核心不在于维度大小，在于标量的数量。 W_Q 的 512 维里包含了主谓、指代、修饰等所有信息，但经过点积全被压缩成一个标量。多头通过切分维度空间，让每个头在有限的 64 维里被迫专业化，因为容量小了，它没法同时照顾所有关系，只能把手头的 64 维做到极致。这就是「子空间专长」的来源。

这些专长不是人为指定的。是训练过程中自然分化出来的。你可以在训练后分析每个头的注意力模式，确实会发现头 1 偏爱主谓、头 2 偏爱指代，但没有人会在训练前告诉它们该学什么。

一个句子，8 个视角

句子：「昨天小明把他新买的手机忘在了图书馆」

头 1，主谓一致：「小明」→「忘」，确认动作主体。

头 2，指代消解：「他」→「小明」，不是随便一个路人。

头 3，修饰关系：「新买的」→「手机」，修饰语不是修饰「图书馆」。

头 4，局部邻接：「手机」←「忘」，相邻词形成动宾搭配。

头 5，长距离依赖：「昨天」（句首）→「忘」（句中），时间状语修饰。

头 6，介词结构：「在」→「图书馆」，地点标记。

头 7，标点/边界：句子首尾位置识别。

头 8，语义角色：「小明」施动、「手机」受动、「图书馆」地点。

W_O 把这 8 个 64 维拼成的 512 维再揉一遍，做一个跨视角的融合，「主谓关系说小明是主语」+「指代关系确认他就是那个他」→最终确定「小明」是整个事件的施动者。

磨平一些信息差。

第 9 课：位置编码，模型怎么知道词顺序

Attention 公式里的 QK^T 是点积，点积是可交换的。 $Q_{\text{猫}} \cdot K_{\text{坐}} = Q_{\text{坐}} \cdot K_{\text{猫}}$ 。这意味着「猫追老鼠」和「老鼠追猫」在 Attention 眼里是完全一样的计算，算不出谁追谁。必须额外注入位置信息。

绝对位置编码：给每个座位贴门牌号

原版 Transformer 的做法：给位置 0 编一个 512 维向量，位置 1 编另一个 512 维向量，直接加到 Token 的 Embedding 上，然后正常做 Attention。位置编码用 sin 和 cos 函数生成：

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d}), \quad \text{PE}(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{2i/d})$$

偶数维度用 sin，奇数维度用 cos。 i 是维度索引， pos 是位置编号， d 是总维度 (512)。 $10000^{2i/d}$ 是一个随维度递减的频率，低维度变化慢（捕捉全局位置），高维度变化快（捕捉局部位置）。

致命伤是外推。训练时 pos 最大只到 511（序列长度 512），模型学会了「位置 511 的 cos 值大约是 0.62」。推理时序列长度 1024，位置 1023 的 cos 值是模型从未见过的数值，它不会处理。像乘法口诀表只背到 9×9 ，碰到 10×10 只能瞎猜。

相对位置编码：不算绝对位置，算谁离谁多远

换一种思路。不关心 token 在第几位，关心「两个 token 之间隔了多远」。距离 3 在句首还是句尾都一样。学一个距离到偏置的映射表：

$$\text{Attention_Score}(i,j) = Q_i \cdot K_j + \text{Bias}_{|i-j|}$$

$|i-j|=1$ 有一个可学习的偏置参数， $|i-j|=2$ 有另一个，一直到最大距离 k 为止。超过 k 的距离共享同一个偏置。

优势是平移不变，「猫坐垫子」不管在一篇文章的第 1 句还是第 100 句，内部距离关系一样。劣势是超过 k 的距离全被一视同仁（距离 500 和距离 5000 没有区别），而且参数表随 k 增长。

RoPE：不用加也不用学，直接旋转

当前最优方案。不把位置信息「加」到向量上，而是用旋转矩阵「乘」到 Q 和 K 上。

核心操作：把 Q 向量的每两个相邻维度看作平面上的一个点 (x, y)，然后用位置 m 乘以频率 θ 的角度旋转这个点。

$$R_m = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}$$

不同维度对用不同的频率 θ 。 $\theta_i = 10000^{-2i/d}$ ，高维度转得快（对近距离敏感），低维度转得慢（对远距离敏感）。

旋转后计算 Attention 分数，奇迹发生了：

$$Q_m \cdot K_n = (R_m q)^T (R_n k) = q^T R_m^T R_n k = q^T R_{n-m} k$$

旋转矩阵的群性质： $R_m^T R_n = R_{n-m}$ 。两个旋转的复合等于角度差对应的旋转。所以内积的结果只依赖相对位置 (n-m)，跟绝对位置 m、n 无关。

位置 5 和位置 8 做 Attention，和位置 1005 和 1008 做 Attention，结果完全一样，都是旋转角 3。这就是外推强的根源：新位置只是多转几圈。

Llama、Qwen、DeepSeek 全都用 RoPE。有绝对编码的实现简洁（不需要额外参数表），同时有相对编码的外推能力。

RoPE 怎么工作的，两个词走一遍

句子只有两个词：「猫」（位置 0）、「坐」（位置 1）。 $d=4$ ， $\theta=[1.0, 0.5]$ 。

$$Q_{\text{猫}} = [0.8, 0.6, 0.3, 0.9]$$

位置 0 的旋转： - 维度 (0,1)：没有旋转，角度 = $0 \times 1.0 = 0 \rightarrow [0.8, 0.6]$ 不变 - 维度 (2,3)：没有旋转，角度 = $0 \times 0.5 = 0 \rightarrow [0.3, 0.9]$ 不变

Q_坐 = [0.4, 0.2, 0.7, 0.5]

位置 1 的旋转： - 维度 (0,1)：旋转 $1 \times 1.0 = 1$ 弧度。 $\cos(1) \approx 0.54$, $\sin(1) \approx 0.84$ - 新值：
 $[0.4 \times 0.54 - 0.2 \times 0.84, 0.4 \times 0.84 + 0.2 \times 0.54] = [0.05, 0.44]$ - 维度 (2,3)：旋转 $1 \times 0.5 = 0.5$ 弧度。
 $\cos(0.5) \approx 0.88$, $\sin(0.5) \approx 0.48$ - 新值： $[0.7 \times 0.88 - 0.5 \times 0.48, 0.7 \times 0.48 + 0.5 \times 0.88] = [0.38, 0.78]$

现在 **Q_猫**·**K_坐**，两个向量做内积。因为 **K_坐** 也经过了同样的旋转，内积结果里自动包含了一个「旋转角 = $1 - 0 = 1$ 的相对位置信息」。如果句子变长到位置 100 和 101，旋转角还是 1，结果完全相同。

磨平一些信息差。

第 10 课：残差连接，梯度的高速公路

先理解训练是什么

训练一个神经网络，本质就是调参数。70 亿个参数，每个都是一个数字。训练开始时这些数字是随机的，模型的回答也是瞎蒙的。

训练的过程：给模型看一句话，让它猜下一个词。猜错了，就算一算「猜得有多离谱」（Loss）。然后根据这个离谱程度，反推**每个参数应该往哪个方向调、调多大**。这个「应该往哪调、调多大」的信号，就叫梯度。

梯度就是指南针。它告诉你：这个参数加一点会更好还是减一点会更好。所有参数都跟着自己的梯度微调一下，模型猜下一个词的能力就提升了一点点。重复几十万亿次。

梯度的传递问题

梯度是从最后一层往第一层算的（这叫反向传播）。最后一层紧挨着 Loss，梯度信号又强又准。往前传一层，信号衰减一点。再往前传一层，再衰减一点。

每层都有一个导数（通常小于 1）。梯度经过 100 层之后，就等于原始梯度乘以 100 个小于 1 的数。结果趋近于零。

最前面几层的参数收到的是「应该往哪调」的信号几乎是零，它们不知道该怎么改，干脆不动了。第 100 层学得很好，第 1 层原地踏步。整个模型的表达能力被浅层卡死了。

残差连接的解法

残差连接做的事情极其简单：每一层不只输出 $F(x)$ ，还额外把原始输入 x 原封不动送一份到加法节点。

$$y = x + F(x)$$

这对前向计算没什么特别的，就是多加了一个 x 。

但对梯度的影响是革命性的。反向传播时，梯度不再是「过 100 层每层乘个 <1 的数」。残差路径直接给梯度开了一条**不经过任何运算**的直通车。数学上：

$$\partial y / \partial x = I + \partial F / \partial x$$

前面的 I 是单位矩阵，它不对梯度做任何衰减，保底留下一个完整的原始梯度信号。后面那项可能有衰减，但前面的 1.0 是雷打不动的。

数值走一遍

假设一个 10 层网络，每层的前向计算导数平均是 0.8。

没有残差，纯链式法则：

第 10 层梯度：1.0
第 9 层： $1.0 \times 0.8 = 0.80$
第 5 层： $0.8^5 = 0.33$
第 1 层： $0.8^9 = 0.13$

第 1 层收到的梯度只剩 13%。几乎调不动参数。

有残差， $dy/dx = 1.0 + 0.8 = 1.8$ ：

第 10 层梯度：1.0
第 9 层： $1.0 \times 1.8 = 1.80$
第 5 层： $1.8^5 = 18.9$
第 1 层： $1.8^9 = 198.4$

梯度不仅没消失，反而在增强。每一层至少保留着 1.0 的基础通路，这就是残差网络为什么能堆 100 层以上的秘密。

2015 年 ResNet 用残差连接把 152 层深度跑通了。同期没有残差的网络 30 层就训不动。Transformer 每层有两个残差：Attention 后加一个，FFN 后加一个。两条高速路，永不衰减。

磨平一些信息差。

第 11 课：LayerNorm，为什么每个 Transformer 层里都塞了一个它

归一化在干什么

Transformer 有几十上百层。每层都在做计算，Attention 把数值调来调去，FFN 又调来调去。假设每一层只是轻轻把数值放大了 1.1 倍，100 层之后就是 $1.1^{100} \approx 13780$ 倍。数值爆炸了。反过来如果每层缩小 0.9 倍，100 层之后趋近于零。无论是爆炸还是消失，模型都训不动。

归一化做的事极其简单：不管上一层输出了什么乱七八糟的数值，强行把它拉到「均值 ≈ 0 、标准差 ≈ 1 」的标准格式，再喂给下一层。就像一个翻译官站在每层之间，上一层说分贝，下一层说毫米，翻译官统一翻译成标准普通话。

公式就两步。第一步，减均值除以标准差：把整个数据平移到以 0 为中心、压缩到标准差为 1。第二步，乘一个 γ 再加一个 β ：不是完全锁死，给每层留一点「个性」的空间。 γ 和 β 是训练过程中自己学出来的。

BN 和 LN 的区别，只是统计范围不同

公式完全一样。唯一区别是计算均值 μ 和标准差 σ 的时候，从哪些数据里取样。

数据在 GPU 里的形状是三维的：batch（一批几条句子）、seq_len（每条句子几个 Token）、features（每个 Token 是多少维）。比如（32 句，100 Token/句，768 维）。

BatchNorm 的做法：固定一个维度（比如第 0 维），跨那 32 个句子、跨那 100 个 Token，一起算均值和标准差。等于把 $32 \times 100 = 3200$ 个值拉进一个池子统计。这池子足够大，统计量很稳定。

LayerNorm 的做法：只看一个句子里的某一个 Token。在这个 Token 的 768 个维度内部算均值和标准差。不跨句子，不跨 Token。一个 Token 自己的 768 个数跟别的 Token 无关。

	BatchNorm	LayerNorm
统计范围	跨所有句子、跨所有 Token	只在当前 Token 的维度内
pool 大小	$32 \times 100 = 3200$ 个值	768 个值
有多少个 μ, σ	768 对（每个维度一对）	$32 \times 100 = 3200$ 对（每个 Token 一对）

为什么图像用 BN，文本用 LN

图像处理的场景：一张图 224×224 像素，batch size 可以开到 128。BN 的统计池于是 $128 \times 224 \times 224 \approx 640$ 万个值，超级稳定。而且所有图片都是等大的，不存在「这张图比那张图短」的问题。BN 在图像领域是标配。

文本处理的场景恰恰相反。每条句子长度不一样，「我爱 NLP」3 个 Token，「今天天气很好我去公园」7 个 Token。短的要补 PAD 占位符到一致长度。如果做 BN，PAD 这个零向量也会被拉进统计池，污染均值和标准差。

更致命的是，NLP 的 batch size 经常很小。一条长文本可能就占满一张 GPU 的显存，batch size 可能是 2 或 4。BN 用 $2 \times 100 = 200$ 个值去估计一个维度的均值，统计极不稳定。

LN 不跨样本不跨 Token。即使 batch size=1、句子只有 4 个 Token，每个 Token 的 768 个维度足以算出一个可靠的统计量。而且推理时不需要维护训练时的滑动平均，每次直接从当前 Token 的特征内部算，训练和推理完全一致。

这就是为什么 Transformer 的每一层都塞了一个 LayerNorm。Llama 和 DeepSeek 甚至用了更省的 RMSNorm，只做方差归一化，不管均值，省了均值的计算量，效果基本持平。

走一遍：一个 5 维向量

某层 Attention 输出 $[2.0, -1.5, 3.2, -0.8, 1.1]$ 。

Step 1，算均值： $\mu = (2.0 - 1.5 + 3.2 - 0.8 + 1.1) / 5 = 0.8$

Step 2，算标准差： $\sigma = \sqrt{[(1.2^2 + (-2.3)^2 + 2.4^2 + (-1.6)^2 + 0.3^2) / 5]} = \sqrt{(15.14/5)} \approx 1.74$

Step 3，归一化：每个元素做 $(x - 0.8) / 1.74$ 。得到 `[0.69, -1.32, 1.38, -0.92, 0.17]`。均值 0，标准差 1。

Step 4， γ 和 β 放回来（假设 γ 全是 1.2， β 全是 0.1）：`[0.93, -1.48, 1.76, -1.00, 0.30]`

γ 和 β 是 LayerNorm 保留的自由度。完全锁死在 $[0,1]$ 范围会丢掉有用的幅值信息。 γ 和 β 让归一化后的向量保留一定的「个性」。

磨平一些信息差。

第 12 课：Pre-norm vs Post-norm

这个问题怎么来的

第 10 课讲了残差连接，每层输出 $= x + F(x)$ ，给梯度一条直通高速路。第 11 课讲了 LayerNorm，把数值拉到标准分布，防止爆炸和消失。

Transformer 的每一层里，这两个东西都要用到。于是一个很自然的问题出现了：**残差连接里的加号和 LayerNorm，谁先谁后？**

这个问题看起来微不足道。一个加号放在归一化前面还是后面，能有多大事？但在 2017 年原论文和之后所有的现代模型之间，恰好就是这个顺序不一样。而且原论文选错了。

原论文的做法（Post-norm）：先加，再归一化

$$y = \text{LayerNorm}(x + F(x))$$

数据流是这样的：残差连接先把 x 和 $F(x)$ 加起来，然后整体送进 LayerNorm 做归一化。

问题出在梯度回传上。第 10 课说过，残差连接之所以能防梯度消失，是因为梯度有一条**不经过任何运算的直通道**。但 Post-norm 把这个直通道给堵了， x 的梯度必须先通过加号，再穿过 LayerNorm 才能回到 x 。LayerNorm 里有除法（除以标准差 σ ），这就把直通道变成了弯通道。

训练初期，网络各层的输出还不稳定，标准差 σ 忽大忽小。 σ 小的时候（比如 0.1），梯度通过 LayerNorm 时被放大 10 倍，参数一步跨出去老远，训练炸了。 σ 大的时候（比如 10.0），梯度被压缩到 0.1 倍，参数几乎不动，学不了任何东西。

这就是为什么原论文需要 Warmup：训练前几千步必须用极小的学习率，等各层的 σ 稳定在一个合理范围了，再慢慢把学习率加回正常值。Warmup 的步数设错了训练就崩，是一个非常脆弱的工程补丁。

现代做法（Pre-norm）：先归一化，再加

$$y = x + F(\text{LayerNorm}(x))$$

LayerNorm 被挪到了 $F(x)$ 的**前面**。这意味着残差连接绕过了 $F(x)$ ，也绕过了 LayerNorm。

梯度回传时，从 y 回到 x 有两条路。一条穿过 $F(\text{LayerNorm}(x))$ 的路径可能有衰减。但另一条是残差的**纯直通线**， x 的梯度从终点直接跳回起点，不经过任何运算。数学上 $\partial y / \partial x = 1 + \partial F(\text{LN}(x)) / \partial x$ ，前面的 1 就是这条直通线给的保底。

训练从一开始就是稳定的。不需要 Warmup。第一步就能用正常学习率。

代价和选择

Pre-norm 有一个微小的代价：LayerNorm 在 $F(x)$ 之前做归一化，等于每一层的输入总是被削平的。这在理论上可能轻微削弱深层网络的表达能力，因为归一化抹掉了一些幅值信息。

但 100 层 Transformer 训不训得动，远比这点理论上的表达损失重要。所有现代模型，GPT、Llama、DeepSeek、Qwen，全部用的 Pre-norm。

Post-norm 已经被工程实践淘汰了。不是因为理论上 Pre-norm 效果更好，而是因为 Pre-norm **不需要那个脆弱得令人发指的 Warmup**。

磨平一些信息差。

第 13 课：两种 Mask，为什么模型必须戴眼镜

先理解一个问题：Attention 是盲的

第 7 课讲了 Attention 的核心操作， QK^T 做矩阵乘法，每个 Token 对所有 Token 算相关性分数。这个操作本身没有任何限制。它不知道哪些 Token 是真实语义，哪些 Token 是填充占位符。它也不知道生成时当前位置后面的词还不存在。

如果不加 Mask，模型会给 PAD 位置分配一大堆注意力（浪费）、训练时预测下个词就偷看了标准答案（作弊）、推理时想看未来看不到（训练推理 mismatch）。

Mask 的作用就是在这张全连接的注意力矩阵上画一张准入地图，哪行哪列允许互相看，哪行哪列屏蔽掉。

第一种 Mask：Padding Mask

训练不能一条一条喂句子，太慢了。必须把多条句子拼成一个 batch 同时喂给 GPU。但句子长度不一样：「我爱 NLP」有 3 个 Token，「今天天气很好我去公园散步」有 8 个 Token。怎么拼成一个整齐的矩阵？

办法是补齐：所有句子补到 batch 里最长那条的长度。短的句子后面补 PAD。PAD 是一个占位符，不包含任何语义信息。

问题来了：PAD 也会进入 Attention 计算。如果不加 Mask，真实的 Token 会分一部分注意力给 PAD，那些注意力本来应该用在有意义的词上，现在浪费在了一个空位上。而且 PAD 自己的 QK^T 结果也会产生 Loss，反向传播时这些 Loss 会传回 PAD 的随机向量，产生无意义的梯度更新。

Padding Mask 的解法：把所有 PAD 位置在注意力矩阵里标记出来，在送入 Softmax 之前把这些位置设为 $-\infty$ 。Softmax 之后 $e^{(-\infty)} = 0$ ，PAD 位置分到的注意力精确归零，等于从计算图里物理消失。

第二种 Mask：Sequence Mask（因果掩码）

第 1 课说了，GPT 类模型是逐 Token 预测下一个词的。训练时为了效率，会把完整句子一次性输入，然后用 Mask 限制每一位置只能看到自己和过去。

为什么要限制？如果训练时模型能看到未来的词，它预测「猫坐在垫子上」里的「上」时，已经能看到后面的句号甚至是第二个句子的第一个词。它学到的不是「根据前文猜后文」，而是「从整段文本里找答案」。但推理时它一次只能生成一个 Token，未来的词**不存在**。训练时学会了依赖未来信息，推理时找不到，这就是曝光偏差（train-inference mismatch）。模型训练时成绩满分，上线后胡说八道。

Sequence Mask 的解法：一个下三角矩阵。对角线及以下全是 1（可以看），对角线以上全是 0（不能看）。处理第 1 个词时只看自己，处理第 3 个词时看前两个和自己，以此类推。训练和推理的输入分布完全一致，两者都只能看到过去。

为什么是 $-\infty$ 而不是 0

如果用 0 做掩码值，Softmax 后 $e^0 = 1$ ，被屏蔽的位置仍然分到了权重（约 20%）。只有 $-\infty$ 让 $e^{(-\infty)} = 0$ ，权重精确归零。实际代码里用的是 $-1e9$ 或 `float('-inf')`。

走一遍：「猫坐在垫子上」

输入序列：[猫, 坐, 在, 垫子, 上]，假设 batch 中最长句子是 7，补齐两个 PAD。

Padding Mask（1=允许，0=屏蔽）：

猫	坐	在	垫	上	P	P	
1	1	1	1	1	0	0	猫看全部真实词
1	1	1	1	1	0	0	坐看全部
1	1	1	1	1	0	0	在看全部
1	1	1	1	1	0	0	垫子看全部
1	1	1	1	1	0	0	上看全部
0	0	0	0	0	0	0	PAD 行全屏蔽
0	0	0	0	0	0	0	PAD 行全屏蔽

PAD 的行和列全屏蔽，既不查别人，也不被别人查。

Sequence Mask（Decoder 自注意力）：

猫	坐	在	垫	上	
1	0	0	0	0	猫只看自己
1	1	0	0	0	坐看猫+自己
1	1	1	0	0	在看前两个+自己
1	1	1	1	0	垫子看前三个+自己
1	1	1	1	1	上看全部历史

对角线以上全 0，不能看未来。

两个 Mask 合并（逻辑 AND，任一屏蔽就屏蔽）：

猫	坐	在	垫	上	P	P	
1	0	0	0	0	0	0	猫只看自己
1	1	0	0	0	0	0	坐看猫+自己
1	1	1	0	0	0	0	在看猫+坐+自己
1	1	1	1	0	0	0	垫子看前三个+自己
1	1	1	1	1	0	0	上看全部历史
0	0	0	0	0	0	0	PAD 全屏蔽
0	0	0	0	0	0	0	

Encoder 只加 Padding Mask（双向注意力，不需要因果限制）。Decoder 自注意力两个都加。Decoder 的 Cross-Attention 看的是 Encoder 的输出，只需要 Padding Mask，因为 Encoder 的输出是已经算完的静态数据，不存在「未来」的问题。

磨平一些信息差。

第 14 课：参数 vs 数据，容器和内容

GPT-3 有 1750 亿参数，但只读了 3000 亿 Token。Chinchilla 只有 700 亿参数，却读了 1.4 万亿 Token。结果 Chinchilla 全面碾压。

参数是容量，数据是营养。大容量配不上足够的营养，就像 2 米的篮球运动员每天只吃一顿饭。

Chinchilla 论文给出经验公式：最优比例 $D \approx 20 \times N$ 。每 1 个参数喂约 20 个 Token。大多数早期大模型都是「大但没吃饱」，参数量巨大但数据量严重不匹配。

后续 Llama 用远超 Chinchilla 比例的数据训练，因为高质量数据可以多 epoch 重复使用，不严格遵守「只训一轮」假设。但 Chinchilla 的核心理念仍然成立：数据和参数要匹配增长。

Chinchilla 怎么算出 20:1 这个比例

固定算力预算 $C = 1000$ PFlops。Chinchilla 训了十几组 (N, D) 组合：

N (参数)	D (Token)	$C \approx 6ND$	Test Loss
2B	200B	$2.4e21$	2.85
4B	400B	$9.6e21$	2.62
7B	1.4T	$5.9e22$	2.45 ← Chinchilla 最优
10B	600B	$3.6e22$	2.68
70B	300B	$1.3e23$	2.91 ← Gopher 的配置

Chinchilla 70B + 1.4T tokens 用跟 Gopher 280B 一样的算力，Loss 低了将近 0.5。不是参数越多越好，而是在给定的 C 下找到 N 和 D 的最优平衡点。

公式： $N_{\text{opt}} \propto C^{0.5}$ ， $D_{\text{opt}} \propto C^{0.5}$ 。算力翻 4 倍，参数翻 2 倍、数据也翻 2 倍。两者对半增长，不是一边倒。

磨平一些信息差。

第 15 课：MLM vs CLM，两种截然不同的教法

MLM (Masked Language Model)：随机遮 15% 的词，模型根据前后文猜。BERT 的训练方式。优势是双向理解，猜被遮住的词需要同时看前后。劣势是只有 15% 的 Token 产生学习信号，且推理时没有 [MASK]，存在训练推理 mismatch。

CLM (Causal Language Model)：逐词预测下一个。GPT 的训练方式。每一个 Token 都是学习信号，训练和推理完全一致，都是预测下一个词。天然适合生成任务。劣势是单向，不能同时利用前后信息。

理解任务选 MLM。生成任务选 CLM。现代 LLM 全用 CLM，不是为了理解更好，是为了生成更强。

同一段文字，MLM 和 CLM 各能学出多少信号

训练语料：「今天天气很好，我和小明去公园散步」

CLM 方式，7 个 Token 产生 7 个预测任务：

今天 → 预测「天气」
今天 天气 → 预测「很好」
今天 天气 很好 → 预测「，」
...

7 个 Token，7 次学习机会。每个位置都在回答问题。

MLM 方式，随机遮 15%，产生 1 个预测任务：

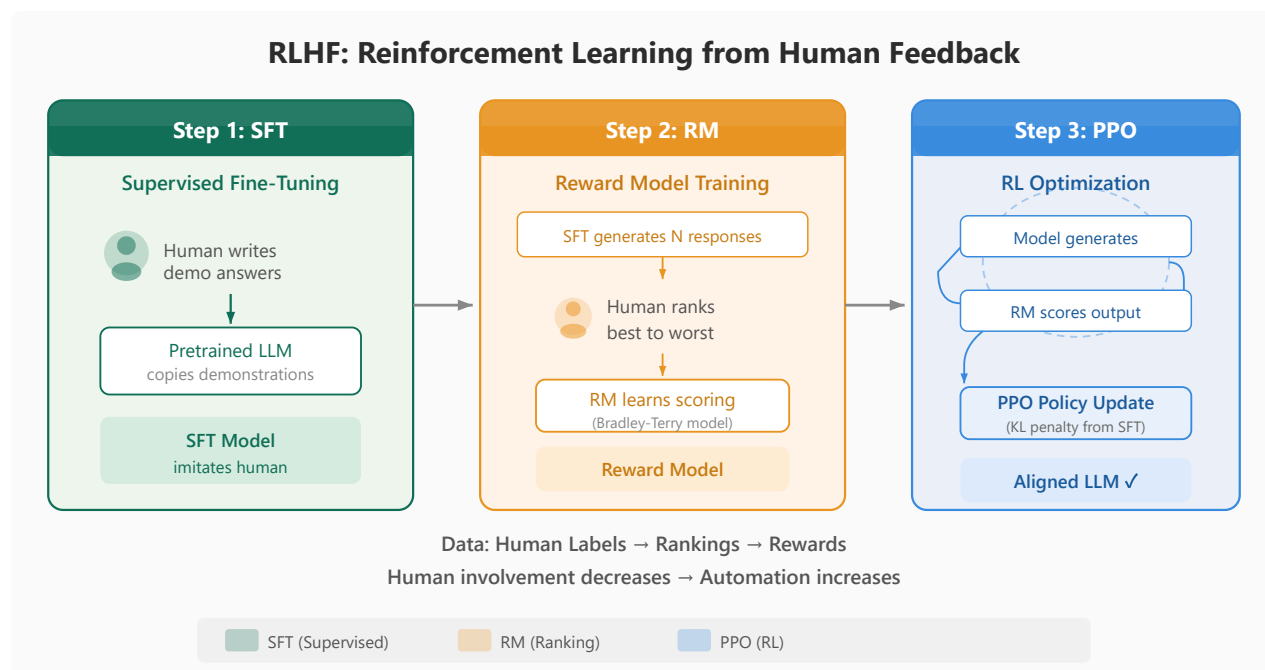
今天 [MASK] 很好，我 [MASK] 小明去公园散步
→ 预测「天气」和「和」

7 个 Token，只有 1 个 MASK token 产生信号（约 15%）。其余 6 个只提供上下文，不产生梯度。

同一段文字，CLM 产出的有效训练信号是 MLM 的约 6-7 倍。这就是 CLM 在互联网海量文本上的数据效率碾压 MLM 的根本原因。

磨平一些信息差。

第 16 课：RLHF 全流程，SFT → RM → PPO



RLHF 三阶段

SFT：人类写几千条「完美回答」，模型用交叉熵损失学着模仿。具体做法：标注员对着 prompt 手写「理想答案」，SFT 阶段就是让模型逐 token 预测标注员写的词。底层算法跟预训练一样（猜下一个词），但数据质量极高。学完能从「会说完整句子」变成「能写出及格水平的回答」。

RM：训练一个独立的打分模型。同一个 prompt，SFT 模型生成 4 个不同回答 (A/B/C/D)，人类标注员排序 ($A > B > C > D$)，不打绝对分。为什么排序而非打分？因为人与人之间的绝对打分尺度差异太大，有人给 5 分极其慷慨，有人很苛刻。但所有人都能判断「A 比 B 好」。RM 用这个排序数据学习一个标量输出：给定 (prompt, response)，输出分数 r 。分数越高的回答越符合人类偏好。

PPO：策略模型（SFT 初始化）生成回答 → RM 打分 → 用强化学习算法更新策略参数 → 下一轮。反复循环直到模型稳定输出高质量回答。

PPO 到底在算什么

PPO 的核心操作是控制「新旧策略的差距」。策略 = 给定 prompt 下，模型输出各个 token 的概率分布。

定义概率比： $r_t(\theta) = \pi_\theta(a_t/s_t)/\pi_{\theta_{old}}(a_t/s_t)$

分子是新策略在状态 s_t 下输出动作 a_t 的概率。分母是旧策略在同一状态输出同一动作的概率。 r_t 接近 1 表示新旧策略差不多，偏离 1 表示策略变化太大。

PPO 的损失函数：

$$\text{Loss} = -\min(r_t \cdot A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \cdot A_t)$$

A_t 是优势函数，表示这个动作比平均水平好多少（RM 给出的奖励减去基准线）。

如果 $A_t > 0$ （这个动作好，应该多做），PPO 鼓励 r_t 变大，但不能超过 $1+\epsilon$ （通常 $\epsilon=0.2$ ）。超过之后梯度被截断，防止一次更新步子太大。

如果 $A_t < 0$ （这个动作差，应该少做），PPO 鼓励 r_t 变小，但不能低于 $1-\epsilon$ 。低于之后同样截断。

这就像限速：策略可以优化，但每次最多偏 20%。不能因为某个回答 RM 给了一次高分就把整个策略砸过去学它。

KL 散度约束和 Reward Hacking

PPO 的 Clip 机制控制单步更新幅度，但多轮累积后策略仍然可能慢慢「漂移」远离 SFT。所以额外加 KL 散度约束。

$$\text{Objective} = \mathbb{E}[r] - \beta \cdot \text{KL}(\pi_{\text{new}} \parallel \pi_{\text{SFT}})$$

KL 散度度量两个概率分布的差异。SFT 输出「苹果」的概率是 0.3，新策略输出是 0.9，KL 就会变大。 β 是惩罚系数（通常 0.01-0.1）。

一个真实案例：RM 从 Reddit 点赞数据训练，学到了「回答越长，RM 分越高」。PPO 阶段模型发现这个漏洞，开始在回答后面加礼貌废话，「希望对你有帮助，如有疑问请联系，祝你愉快」，RM 给高分但人类读了烦。

$KL \approx 0.05$ (轻微偏离 SFT) $\rightarrow$ 正常优化

$KL \approx 0.5$ (明显偏离 SFT) $\rightarrow \beta \times 0.5$ 抵消掉大部分 reward

$KL \approx 2.0$ (翻墙) $\rightarrow$ 惩罚大于 reward, PPO 主动拉回来

这张 KL 网把模型锁在 SFT 的合理区域里优化，防它变成讨好 RM 的废话机器。RLHF 同时维护四个模型（策略模型、参考模型、奖励模型、价值模型），这就是它成本高的根本原因。

磨平一些信息差。

第 17 课：DPO 和 GRPO，在 PPO 的基础上做了什么

PPO 的痛苦在哪

第 16 课讲了 RLHF 的 PPO 阶段。如果你没亲自跑过，可能觉得「四个模型」只是数字。实际上，每训练一步 PPO，显存里要同时住着四个庞然大物：

策略模型（你要训的那个）、参考模型（冻结的 SFT，用来算 KL 约束）、奖励模型（给每句话打分）、价值模型（预估当前状态的期望回报，用来算优势函数）。

四个 7B 模型，FP16，光参数就要 $4 \times 14 = 56$ GB。加上梯度、优化器状态，一张 A100 80GB 勉强强。训练时要先让策略模型生成回答，RM 打分，价值模型算优势，参考模型算 KL，然后才能做一次梯度更新。每一步的计算量约等于做四次前向传播。然后 PPO 还要调 Clip 范围 ϵ 、KL 系数 β 、价值模型的学习率、采样的温度.....超参数起来没完。

而且 PPO 是强化学习算法，不是监督学习。RL 训练天然不稳定，奖励信号稀疏、KL 约束可能过紧或过松、策略可能在几轮内突然崩溃输出乱码。训过 PPO 的人都经历过「跑了 500 步好好的，第 501 步模型突然开始无限重复一个词」的绝望。

DPO：既然数学上可以一步到位，为什么还要跑 RL

2023 年斯坦福的 DPO 论文做了一个漂亮的数学推导。他们发现：PPO 的目标函数（最大化奖励时加上 KL 约束）存在一个闭式解。意思是，理论上存在一个最优策略，满足 PPO 想要收敛到的那个状态，而且可以不用强化学习来逼近。

这个闭式解可以直接写成损失函数：

$$L_{DPO} = -\mathbb{E}[\log \sigma(\beta \cdot (\log \pi_{\theta}(y_w|x)/\pi_{ref}(y_w|x) - \log \pi_{\theta}(y_l|x)/\pi_{ref}(y_l|x)))]$$

公式不用全懂。看它做了什么就够：

- y_w 是人类偏好的好回答， y_l 是差回答
- $\pi_{\theta}(y_w|x)$ 是当前策略给好回答的概率。 $\pi_{ref}(y_w|x)$ 是参考模型（SFT）给好回答的概率

- 如果当前策略认为好回答的概率比参考模型高（比值 > 1 ），日志里是正数，加分
- 如果当前策略给差回答的概率反而高，日志里是负数，扣分
- β 控制不要让新策略偏离参考模型太远

DPO 不需要奖励模型。不需要采样。不需要 PPO 循环。 你只需要准备「(prompt, 好回答, 差回答)」三元组，跟训 RM 用同样的偏好数据，然后用这个损失函数做一次普通的监督学习。一个模型、一次前向+反向传播、一步更新。计算量约等于再做一次 SFT。

代价是 DPO 对偏好数据质量极其敏感。PPO 有 RM 做缓冲，RM 是一个独立训练的打分模型，对单一偏好标注的噪声有平滑作用。DPO 直接把偏好信号打到策略上，偏好数据里如果标注员 A 和标注员 B 的观点冲突严重，DPO 的梯度方向就会摇摆。

GRPO：DeepSeek 的务实方案

GRPO（Group Relative Policy Optimization）是 DeepSeek 在 2024 年提出的。想法很务实：PPO 四个模型太贵，DPO 不需要 RM 但完全依赖标注质量。能不能取中间，保留 RM，但砍掉价值模型，用组内相对评分代替绝对评分？

具体做法：同一个 prompt，策略模型生成 8 个回答。RM 给 8 个回答各打一个分：[3.2, 4.1, 2.8, 3.9, 2.5, 4.4, 3.0, 3.7]。算组内均值 $\mu=3.45$ ，标准差 $\sigma=0.62$ 。每个回答的「相对优势」= (分数 - μ) / σ ：

回答 6：(4.4 - 3.45) / 0.62 = +1.53 ← 明显比组内平均好
 回答 1：(3.2 - 3.45) / 0.62 = -0.40 ← 略低于平均
 回答 5：(2.5 - 3.45) / 0.62 = -1.53 ← 明显比组内平均差

这个标准化操作把 RM 的绝对分数偏差给消掉了。RM 可能天生打分偏高（全部在 4.0 以上），或者偏低（全部在 2.0 左右），但「组内谁比谁好」这个相对排序是稳定的。均值减法消掉了 RM 的系统偏差，标准差除法做归一化。

GRPO 还做了一个关键改动：把 KL 约束从 reward 项挪到 loss 项里。PPO 是在奖励里减 KL 项（软约束），GRPO 是直接在 loss 里用硬上限限制策略偏离。效果是策略跑偏的风险更低，对 β 系数的敏感度也更小。

三句话选型

PPO 效果最稳但成本最高（四个模型 + RL 训练），适合预算充足、安全要求高的 C 端产品。DPO 最省（一个模型 + 一次监督学习），适合预算紧张或快速实验，但依赖偏好数据质量。GRPO 取中间（保留 RM + 去价值模型 + 组内评分），适合生成类任务为主、对成本敏感但需要 RM 做质量把关的场景。

磨平一些信息差。

第 18 课：LoRA，穷玩大模型的终极答案

全参微调 7B 模型要 56GB 显存。你只有 24GB。LoRA 只训 0.1% 的参数。

做法：原始权重 W 冻结不动。旁边加两个小矩阵 $B(d \times r)$ 和 $A(r \times d)$ 。 r 通常 8 或 16，远小于 $d(4096)$ 。 $B \times A$ 的结果尺寸跟 W 一样。推理时 $W' = W + B \times A$ ，速度跟原模型完全相同。

数学假设：模型适应新任务时的权重变化 ΔW 是低秩的，变化只发生在少数几个低维方向。实验验证成立。

QLoRA = LoRA + NF4 量化 + 双重量化。7B 模型只需 6-8GB 显存。RTX 3060 也能跑。

动手算一遍：一个 4×4 的权重矩阵， $r=2$

假设原始权重 W 是一个 4×4 矩阵（真实模型是 4096×4096 ，道理一样）。

W (4×4 , 冻结)

$$\begin{bmatrix} 0.5 & 0.3 & 0.1 & 0.8 \\ 0.2 & 0.7 & 0.4 & 0.1 \\ 0.9 & 0.2 & 0.6 & 0.3 \\ 0.1 & 0.5 & 0.2 & 0.7 \end{bmatrix}$$

B (4×2 , 可训)

$$\begin{bmatrix} 0.2 & 0.1 \\ 0.4 & -0.2 \\ 0.3 & 0.5 \\ -0.1 & 0.3 \end{bmatrix}$$

A (2×4 , 可训)

$$\begin{bmatrix} 0.3 & 0.5 & -0.1 & 0.2 \\ 0.1 & 0.6 & 0.3 & -0.1 \end{bmatrix}$$

Step 1: 算 $B \times A = \Delta W$

$B \times A =$

$$\begin{bmatrix} 0.2 \times 0.3 + 0.1 \times 0.1 & 0.2 \times 0.5 + 0.1 \times 0.6 & \dots \\ 0.4 \times 0.3 - 0.2 \times 0.1 & 0.4 \times 0.5 - 0.2 \times 0.6 & \dots \\ 0.3 \times 0.3 + 0.5 \times 0.1 & \dots & \dots \\ -0.1 \times 0.3 + 0.3 \times 0.1 & \dots & \dots \end{bmatrix}$$

$$\Delta W \approx \begin{bmatrix} 0.07 & 0.16 & -0.01 & 0.03 \\ 0.10 & 0.08 & -0.10 & 0.10 \\ 0.14 & 0.45 & 0.18 & 0.01 \\ 0.00 & 0.13 & 0.10 & -0.05 \end{bmatrix}$$

Step 2: 微调后的权重 $W' = W + \Delta W$

$$\begin{aligned} W'(0,0) &= 0.5 + 0.07 = 0.57 \\ W'(0,1) &= 0.3 + 0.16 = 0.46 \\ &\dots \end{aligned}$$

只改了极小一部分。大多数值几乎不变，但这几个低维方向上的调整足够适配新任务。

参数的差距： - 全参微调要更新 W 的全部 16 个值 - LoRA 只更新 $B(8\text{个}) + A(8\text{个}) = 16$ 个值？不对，这里 4×4 太小，看不出优势。换 4096×4096 ： - 全参更新 16,777,216 个值 - LoRA ($r=8$) 更新 $4096 \times 8 + 8 \times 4096 = 65,536$ 个值 - 差了 256 倍。这就是 LoRA 省显存的原因。

训练完把 ΔW 合入 W ，推理零开销。

磨平一些信息差。

第 19 课：过拟合、梯度问题、Warmup、ZeRO

过拟合 — 模型只记住了训练题的答案

过拟合的意思是，模型在训练数据上表现极好，但在没见过的数据上表现很差。就像一个学生把所有模拟题的答案背下来了，模拟考满分，真题一来就懵。

怎么发现？看两条 Loss 曲线。训练 Loss 持续下降，但验证 Loss 不降反升，gap 在拉大。这就是过拟合的经典信号。

为什么会发生过拟合？模型参数太多，但训练数据太少。7B 参数模型，只给 1000 条数据做全参微调，它有足够多的参数去精确背下这 1000 个样本里的每一个标点。但这些记住的细节在新数据上完全没有泛化能力。

怎么解决？三条路。正则化，Dropout 随机把部分神经元置零，模型没法依赖单个神经元，被迫学更鲁棒的模式。权重衰减让所有参数整体偏小，防止参数为了拟合噪声而长得过大。更多数据，训练数据翻倍，过拟合风险砍半。早停，验证 Loss 开始升的时候就停训，别贪。PEFT 天然防过拟合，LoRA 只训 0.1% 的参数，根本没能力去背训练样本里的所有噪声。

梯度问题 — 爆炸和消失

第 10 课讲过，梯度是告诉每个参数「往哪个方向调多少」的指南针。正常情况下梯度值在一个合理范围，参数平稳更新。

梯度爆炸：某个批次的 Loss 特别大，反向传播时梯度也跟着暴增。参数一次更新跨出老远，之前几十万步学来的东西全被冲掉了。模型输出瞬间变成乱码。解法很简单，梯度裁剪，设一个阈值（默认 1.0），如果梯度的总模长超过这个阈值，就把它等比压缩到阈值范围内。等于给每次参数更新设了一个最高限速。

梯度消失：反过来，梯度趋近于零。参数几乎不动，训练停滞了。在 Transformer 里这个问题已经大幅缓解，第 10 课的残差连接给梯度留了一条直通高速路，不会彻底消失。但在特别深的网络（几百层以上）仍然需要关注。解法是残差连接 + 好的参数初始化（让初始数值落在激活函数的线性区，梯度传递效率最高）。

Warmup — 起步时先慢慢走

训练刚开始的时候，模型参数是随机初始化的，各层的输出完全是随机的。Loss 巨大且极不稳定。如果一上来就用全速学习率，巨大的 Loss 产生巨大的梯度，梯度裁剪虽然能拦住不炸，但前几步的更新方向基本是随机的，因为这些随机参数还没来得及学到任何有意义的模式。

Warmup 的做法：学习率从头 0 开始，线性增长到目标值（比如 $5e-5$ ），通常要几百到几千步。这段时间里模型先用极小的步子探索 Loss 的地形，等方向明确之后再加速。第 12 课的 Pre-norm 已经把 Warmup 的必须性大幅降低了，Pre-norm 本身就能稳定训练早期的数值，但大多数实践者仍然会加一个短暂的 Warmup 做保险。

ZeRO — 把每张 GPU 的负重砍到 1/8

训练大模型时，最头疼的不是算不动，是放不下。第 5 课讲的数据并行有致命缺陷：每张 GPU 都要存一模一样的完整模型 + 完整梯度 + 完整优化器状态。8 张卡就是 8 份完全重复的数据。

为什么优化器状态那么大？训练用的 Adam 优化器不只是保存参数值，它还要保存每个参数的历史梯度（一阶矩）和历史梯度的平方（二阶矩）。参数是 14 GB，优化器状态就是 28 GB（两倍）。每张卡参数 14 + 梯度 14 + 优化器 28 = 56 GB。8 张卡 = 448 GB，其中 96% 是冗余，8 张卡存了 8 份一模一样的东西。

ZeRO 的思想：这些冗余不要了。分片。

ZeRO-1，优化器状态分片。8 张卡，28 GB 切 8 份，每人存 3.5 GB。需要别人的片段时通信取。单卡从 56 GB 降到 31.5 GB。

ZeRO-2，梯度也分片。每人负责一块参数的梯度计算，通信时只传自己的片段。降到 17.5 GB。

ZeRO-3，连参数都分片。前向传播时需要哪块参数，从对应的卡上拉过来，用完扔掉。降到 3.5 GB。

一张 40 GB 的 A100，不加 ZeRO 连 7B 全参微调都跑不动（需要 56 GB）。加了 ZeRO-3，3.5 GB 就能跑，省了 94%。

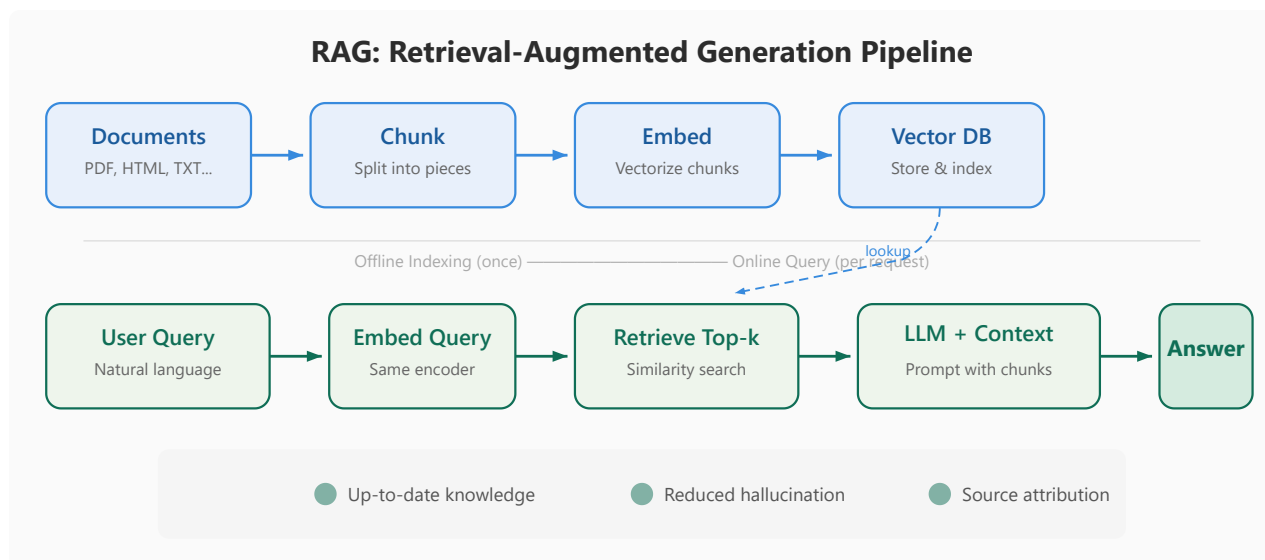
磨平一些信息差。

第 20 课：RAG，让模型会查资料

大模型会编造不存在的 API、不存在的书名、不存在的历史事件。幻觉。

RAG 不给模型凭记忆回答。给它一本参考书，让它翻完再说。

五步。切分：文档切成 200-500 Token 小块。块太大检索不准，太小上下文不够。建索引：每块用 embedding 模型转向量，存进向量数据库。查询：用户问题转向量，检索最接近的 Top-K 块。混合检索：向量检索 + BM25 关键词检索，拼起来重排序，语义相近和精确匹配兼顾。生成：拼成 Prompt 「根据以下资料回答。资料：[检索结果]。问题：[用户问题]」发给大模型。



RAG 流水线

RAG 怎么干掉一次幻觉，完整链路走一遍

用户问：「公司去年 Q3 的销售额是多少？」

Step 1 — 切分：把公司年报切成 300 Token 的小块。财务部分独占 5-6 块，每块标页码和段落号。

Step 2 — embedding：每块过 text-embedding-3-small 转向量，存入 Milvus。

Step 3 — 查询向量化：「公司去年 Q3 的销售额是多少」→ [0.23, -0.45, 0.67, ...] 1536 维。

Step 4 — 检索：向量库返回 Top-5 最相似块。其中 Top-1 匹配到「2025 年第三季度，公司实现销售收入 3.2 亿元」，余弦相似度 0.94。BM25 关键词「Q3」「销售额」也命中同一段。

Step 5 — 生成 Prompt：

根据以下资料回答，资料来源于公司 2025 年年报。

资料 1（相似度 0.94）：「2025 年第三季度，公司实现销售收入 3.2 亿元，同比增长 18.5%

问题：公司去年 Q3 的销售额是多少？

Step 6 — 模型输出：「根据公司 2025 年年报，去年第三季度销售额为 3.2 亿元。」

没有 RAG，模型可能猜「约 2.5 亿」或者编一个数字。有 RAG，它从资料里精确引用。幻觉风险从「可能编」降到了「除非检索失败否则不编」。

磨平一些信息差。

第 21 课：推理优化全家桶

推理和训练的根本区别

训练时，一整段文本同时喂给模型，所有 Token 并行计算。GPU 的几千个计算核心全都在忙。

推理时，每次只能生成一个 Token。生成完「猫」之后才知道下一个是「坐」，生成完「坐」才知道下一个是「在」。GPU 有几千个核心，但一次只算一个 Token，99% 的核心在闲着。TTFT（首 Token 延迟）和 TPOT（每个后续 Token 的延迟）是推理优化的两个核心指标。

五种优化技术各自解决这个问题不同侧面。

KV Cache — 别重复算已经算过的东西

Attention 计算需要 K 和 V。每生成一个新 Token，旧 Token 的 K 和 V 其实没变，「猫」在位置 0 的 K 和 V，在生成「坐」「在」「垫子」时都是一样的值。如果不加 Cache，每一步都要把所有 Token 的 K 和 V 重新算一遍。序列长度 $n=4000$ 时，总计算量是 $O(n^2)$ 。

KV Cache 的做法：算过的 K 和 V 存起来。生成下一个 Token 时，新 Token 的 K 和 V 算一遍，旧的全部从缓存读。计算量从 $O(n^2)$ 降到 $O(n)$ 。代价是 KV Cache 本身占显存，每层每头存一份 K 和一份 V。以 GPT-3 175B 为例，96 层 $\times$ 96 头 $\times$ 2 $\times$ 2048 长度 $\approx$ 2.7 GB。序列越长，Cache 越大。

FlashAttention — 瓶颈不是算力，是搬数据

Attention 的 $O(n^2)$ 矩阵太大，GPU 最快的存储器 SRAM（约 20 MB）装不下，必须存在较慢的 HBM（显存）里。每次做 Softmax 和乘 V，都要从 HBM 把这 $\diamond\diamond$ 大矩阵读出来。GPU 的大多数时间不是在算，是在等数据从 HBM 搬到计算单元。

FlashAttention 的核心操作：不把完整的 $n \times n$ 注意力矩阵写进 HBM。把 Q、K、V 切成小块，每次只搬一小块到 SRAM，在 SRAM 里算完这块的 Attention，直接乘 V 得到部分输出，然后丢掉中间结果处理下一块。

难点是 Softmax 不能分块算，Softmax 需要对一整行的所有分数求和。FlashAttention 用了 online softmax 技巧：边算边维护「当前所见的最大值」和「运行总和」，每处理一个新块就修正之前的输出。结果数学上完全等价于标准 Attention，但 HBM 读写量大幅减少。训练加速 2-4 倍，推理加速 1.5-2 倍。

量化 — 参数精度降一档，显存砍一半

第 5 课讲过。推理时把 FP16 参数量化到 INT8，显存减半。精度损失在大多数场景下不影响输出质量。INT4 再减半，数学推理可能轻微退化，但配合 QLoRA 微调能补回来。

PagedAttention — 让 KV Cache 不产生碎片

传统做法是给每个序列预分配一大块连续显存放 KV Cache。问题：不同序列的长度差异巨大，有的用户只问一句话，有的扔一本《三体》进去。预分配短了序列放不下，预分配长了大量显存浪费。而且序列长度动态增长，移动 Cache 很贵。

vLLM 的 PagedAttention 把 KV Cache 像操作系统内存分页一样管理：切成固定大小的页（比如 16 Token 一页），需要时分配，用完回收。不再需要预分配大块连续内存。碎片化大幅减少，同样一张 GPU 下能同时服务的用户翻几倍。

投机解码 — 小模型先猜，大模型验证

大部分 Token 很容易预测，「今天天气真」后面几乎是 100% 的「好」。让 70B 的大模型去猜这些简单 Token 是浪费。投机解码的做法：用一个极小的草稿模型（比如 0.5B）快速生成几个候选 Token，然后用大模型一次性验证这一串。全部猜对，大模型一次前向传播就确认了好几个 Token，省了重复计算。猜错了，丢掉重来。

能提速多少取决于任务。低熵任务（代码、翻译、固定格式文本）提速 2-3 倍，因为草稿模型几乎全对。高熵任务（开放式写作、创意生成）草稿模型经常猜错，没什么效果甚至更慢。

KV Cache 到底省了多少，一个序列走一遍

序列「今天天气很好」已生成 5 个 Token。

没有 KV Cache，每一步重算全部：

Token 1:	算 K1, V1	(1 次)
Token 2:	算 K1, V1, K2, V2	(2 次)
Token 3:	算 K1-K3, V1-V3	(3 次)
Token 4:	算 K1-K4, V1-V4	(4 次)
Token 5:	算 K1-K5, V1-V5	(5 次)

总共 15 次。序列 4000 时, $O(n^2) \approx 1600$ 万次。

有 KV Cache:

Token 1:	算 K1, V1, 存缓存
Token 2:	读 K1, V1 → 只算 K2, V2
Token 3:	读 K1-K2, V1-V2 → 只算 K3, V3
...	

每个新 Token 只算自己。序列 4000 时, $O(n) \approx 4000$ 次。代价: GPT-3 175B, 96 层×96 头×2×2048 长度 ≈ 2.7 GB 显存被 KV Cache 吃掉。

磨平一些信息差。

第 22 课：AI Agent，让模型会干活

Chatbot 是被动回答，你问一句它答一句，答完就忘。Agent 是主动完成任务，感知环境、做决策、调工具、看结果、调整下一步，循环直到目标达成。

Function Calling：模型的「遥控器」

Agent 最基础的能力是调用外部工具。机制叫 Function Calling：

1. 提前定义好工具清单（函数名、参数格式、功能描述），写在 system prompt 里
2. 用户输入后，模型决定要不要调工具。要调的话，输出一串 JSON 描述「调哪个函数、传什么参数」
3. 外部框架截获这串 JSON，执行真正的函数调用（查数据库、调 API、读文件）
4. 把返回结果追加到对话历史里，喂回给模型
5. 模型基于工具返回结果继续推理或输出最终回答

模型自己不能上网、不能查数据库、不能发微信。它只会生成 JSON。是外部框架在替它执行。这种设计让模型的能力可以无限外挂，加一个新工具只需要在清单里多写一行描述。

ReAct：Agent 的决策循环

ReAct = Reasoning + Acting。最主流的 Agent 决策模式。不是「想好了再动手」，而是「想一步、做一步、看结果、再想下一步」。

一个完整的 ReAct 循环：

Thought（思考）：我现在知道了什么，下一步该做什么
Action（行动）：调用工具
Observation（观察）：工具返回了什么
→ 回到 **Thought**，基于新信息继续

这套循环让 Agent 具备了两个 Chatbot 没有的能力。多步推理：复杂任务不能一步完成，Agent 自己拆成若干步，每一步基于上一步的结果调整。错误恢复：工具返回错误时，Agent 换参数重试或换工具，不会直接输出一个错的最终答案。

一个 Agent 的实际执行过程

任务：「帮我查一下北京明天天气，如果下雨就给我的微信发条提醒」

Thought 1：「先查天气。」 Action 1：生成 Function Call

```
{"function": "get_weather", "parameters": {"city": "北京", "date": "2026-07"}}
```

Observation 1: `{"weather": "中雨", "temp": "26°C"}`

Thought 2：「中雨，需要提醒。」 Action 2：生成 Function Call

```
{"function": "send_wechat", "parameters": {"to": "self", "message": "明天北"}}
```

Observation 2: `{"status": "sent"}`

Thought 3：「搞定了，回复用户。」 Final Answer：「明天北京中雨 26°C，微信提醒已发送。」

Chatbot 只能输出「明天北京可能下雨」。Agent 真的调了 API 查天气，并且真的发了微信。

单 Agent vs 多 Agent

单 Agent 手里有所有工具，自己决策全过程。简单直接，适合任务边界清晰、工具不超过 10 个的场景。能力受限于单个模型的推理深度。

多 Agent 各司其职。一个检索 Agent 从数据库拉数据，一个分析 Agent 做结论，一个写作 Agent 出报告。Agent 之间通过消息传递信息。复杂度指数增长，通信死锁、等待链、谁来仲裁，需要在架构层面做超时和容错。

Google 的 A2A 协议就是为多 Agent 互操作设计的。第 23 课详讲。

磨平一些信息差。

第 23 课：MCP / A2A / Function Call，让模型会叫外援

模型为什么需要外援

大模型的知识截止于训练数据，2024 年之后发生的事它不知道。它不会查数据库，不会发邮件，不会访问 Google Drive。所有能力都被锁在训练语料里。

要让模型超越自己的训练数据，必须给它连接外部工具。四种方案，从简单到复杂。

Function Call — 最原始的工具调用

做法很简单。提前定义工具清单（函数名、参数格式、功能描述），塞进 system prompt。用户提问后，模型决定要不要调工具。要调的话，不输出自然语言，而是输出一段描述性的结构化文本，通常就是 JSON：「调 query 这个函数，sql 参数是 SELECT SUM(amount) FROM orders...」。

外部框架截获这段 JSON，执行真正的 SQL 查询，把查出来的 `[{"sum": 456.80}]` 追加到对话历史里喂回给模型。模型基于真实数据继续推理，输出最终回答。

Function Call 的问题：每个 AI 应用自己定义工具格式。你用 OpenAI 的 Function Call 格式写了一组工具，换到 Claude 就得重写。没有统一标准意味着生态割裂，每个平台一个格式，工具开发者要适配 N 套接口。

MCP — 把 Function Call 标准化

Anthropic 2024 年发布了 MCP（Model Context Protocol）。核心思想：定义一套通用的客户端-服务器协议，让所有工具按同一种方式暴露给所有 AI 应用。

AI 宿主（比如 Claude Desktop、CodeBuddy）是 MCP 客户端。每个工具（PostgreSQL、GitHub、Google Drive）封装成 MCP 服务器。服务器暴露统一的接口规范：列出自己能干什么、每个功能需要什么参数、执行后返回什么格式。

一次开发，所有支持 MCP 的应用都能用。社区已经有数百个现成的 MCP 服务器，数据库查询、文件操作、API 调用、代码执行。开发者不需要为每个 AI 平台各写一套工具适配层。

MCP 的另一个关键设计：权限和资源管理。服务器在启动时声明自己需要什么权限，用户确认后才能连接。不像 Function Call 那样无状态的每次调用，MCP 维持持久连接和资源生命周期。

实际走一遍：MCP 查数据库

假设 Claude 要通过 MCP 查 PostgreSQL。

启动时：MCP 客户端连接 PostgreSQL MCP 服务器。服务器暴露能力清单：

```
{
  "tools": [
    {
      "name": "query",
      "description": "执行 SQL 查询",
      "parameters": {
        "sql": {
          "name": "list_tables",
          "description": "列出所有表"
        }
      }
    }
  ]
}
```

用户提问：「查一下 user_id=1088 昨天的订单总金额」

模型生成调用：不是直接写 SQL，而是按 MCP 接口格式输出调用请求。外部框架截获后交给 MCP 服务器执行 SQL，返回 `[{"sum": 456.80}]`。模型基于真实数据回答。

整个过程，模型不需要知道数据库地址、账号密码、表结构。MCP 服务器做了封装。换成一个 MySQL 服务器，只要实现了相同的 MCP 接口，模型侧零改动。

A2A — 不是模型调工具，是 Agent 找 Agent

MCP 解决的是「模型 ↔ 工具」的问题。但 2024 年 Google 提出了一个不同的场景：两个 Agent 之间怎么通信？

场景：采购 Agent 需要确认供应商库存，它需要联系供应商自己的库存 Agent。这两个 Agent 分别是不同公司开发、部署在不同服务器上的独立系统。它们之间没有统一的通信方式。

A2A（Agent-to-Agent）就是解决这个问题的协议。它定义了 Agent 之间的标准交互流程：发现（找到对方）、握手（确认双方身份和能力）、任务分配（把子任务发给对方）、结果回传。不是模型调一个函数，而是两个自主系统之间的业务协作。

A2A 和 MCP 不冲突。MCP 是「模型的手」，伸手去拿一个外部工具。A2A 是「Agent 的电话」，打电话给另一个 Agent 商量怎么协作。同一个系统里可以同时存在：Agent 通过 MCP 查数据库，通过 A2A 通知另一个 Agent 更新库存状态。

Skill — MCP 之上的编排层

MCP 暴露的是原子工具：执行 SQL、读文件、发 HTTP 请求。但实际任务很少是单一原子操作，「帮我分析上周的销售数据然后发邮件给老板」要分五步，涉及三个不同的工具。

Skill 是一段预定义的 Prompt + 多步工具调用流程。它把 MCP 的原子操作编排成场景化的能力。比如「数据分析 Skill」内部调用 MCP 的 PostgreSQL 查询 + Python 执行 + 邮件发送三个工具，统一封装成一个能力入口。用户只需要描述想分析什么，Skill 自动调度底层的 MCP 工具完成多步操作。

磨平一些信息差。

第 24 课：前沿架构，MoE、GQA、DeepSeek 怎么玩的

MoE — 让模型又大又快

传统 Transformer 的 FFN 层是密集的，每个 Token 都要经过整个 FFN 矩阵的全部参数。7B 模型就是 7B 个参数全部激活一次。模型越大，推理越慢。

MoE（混合专家）换了一种思路：不把 FFN 做成一个大矩阵，而是做成多个「专家」小矩阵。每个 Token 到这一层时，先过一个小路由网络，选择最合适的 Top-K 个专家，只用这几个专家处理这个 Token。其他专家不动。

DeepSeek-V3 的做法：256 个专家，每个约 6.4B 参数。每个 Token 只激活 Top-8。总参数 1.6T，但每次推理的计算量相当于一个 49B 的密集模型，参数量是 GPT-4 量级，推理成本跟 Llama-70B 差不多。

一个 Token 穿过 MoE 层：

Token 「猫」→ 路由网络打分 →

专家 17：0.83（激活）

专家 89：0.72（激活）

专家 3：0.15（忽略）

...

专家 201：0.08（忽略）

最终输出 = $0.83 \times \text{Expert17}(\text{Token}) + 0.72 \times \text{Expert89}(\text{Token})$

路由网络的权重和主模型一起训练。没有人为指定「专家 17 负责动词、专家 89 负责名词」，这些专长是训练过程中自然分化的。

负载均衡：如果所有 Token 都砸向专家 17 和 89，其他 254 个专家白训了，而且那两张 GPU 被挤爆，别的 GPU 闲置。MoE 必须加辅助损失函数惩罚专家使用不均：

$$L_{\text{balance}} = \alpha \times \sum (\text{每个专家被选中的平均概率} - 1/N_{\text{experts}})^2$$

这保证在几十亿 Token 的训练过程中，256 个专家各自的利用率逐渐趋于均匀。

GQA — KV Cache 的信息瓶颈

第 21 课讲了 KV Cache 占显存的问题。Cache 的大小跟注意力头的数量成正比，头越多，Cache 越大。

标准多头注意力 MHA：Q、K、V 都是完整的 H 个头。比如 $H=32$ ，每层存 32 个 K 向量和 32 个 V 向量，显存开销大。但精度高，每个头有独立的 K、V 表达。

MQA (Multi-Query Attention)：K 和 V 砍到只剩 1 个头，所有 Q 头共享。KV Cache 直接砍到 $1/H$ 。省显存省得狠，但信息瓶颈明显，32 个 Q 头只能查同一组 K、V，表达能力受限。

GQA (Grouped Query Attention) 取折中：K 和 V 分 G 组 (比如 $G=8$)，每组内的 Q 头共享 K、V。Q 保持 32 头全量，但 KV 只有 8 组。KV Cache 是 MHA 的 $1/4$ ，表达能力比 MQA 好得多。Llama 2、Qwen 都用的 GQA。

MLA — DeepSeek 的超长文本秘籍

MLA (Multi-head Latent Attention) 是 DeepSeek 对 GQA 的进一步改进。核心操作：K 和 V 不直接存在注意力头维度，而是先投影到一个极小的潜在空间 (比如 64 维)，推理时再从潜在空间恢复到完整的头维度。

这样做的好处：KV Cache 不再随头数增长，而是固定在潜在空间的大小。 $H=128$ 的标准多头注意力，KV Cache 要存 128 组向量。MLA 只需要存 1 组低维潜在向量，Cache 大小只有传统方案的几分之一甚至十几分之一。

这就是 DeepSeek-V3 能支持 128K 长上下文的根本原因。如果按传统的 KV Cache 方案，128K 序列的 Cache 要几十 GB，一张 GPU 根本放不下。MLA 把 128K 的 Cache 压到了传统方案 8K-16K 的水平。

三件套如何组合

DeepSeek-V3 的架构 = MoE (256 专家，每次激活 8 个) + GQA (KV 分组共享) + MLA (KV 潜在空间压缩)。MoE 降低推理计算量，GQA 降低 KV Cache 的显存占用，MLA 进一步把长序列的 Cache 压缩到极致。三件套协同，才能让一个 1.6T 参数的模型在适中的 GPU 集群上跑 128K 上下文。

磨平一些信息差。

第 24.5 课：多模态/VLM 入门，模型怎么看图

到目前为止讲的都是文本模型。但 2025 年之后，面试里越来越多问多模态。GPT-4V、Gemini、Qwen-VL 都能看图了。怎么做到的？

CLIP：图文的翻译官

2021 年 OpenAI 发布了 CLIP，核心操作极其简单：用对比学习让图片和文字在同一个向量空间里对齐。

训练数据是 4 亿对（图片，文字描述）。batch 里混着放，目标是让配对的图文向量距离最近，不配对的推远。训练完后，一张猫的图片拉成的向量，和「一只猫」这行文字拉成的向量，在 512 维空间里几乎重叠。和「一辆汽车」的向量则隔得很远。

CLIP 的结构：一个视觉编码器把图片压缩成 512 维向量，一个文本编码器把文字也压缩成 512 维向量。两个向量做余弦相似度。它是目前所有 VLM 的地基。

VLM 怎么把图喂给 LLM

标准架构分三步。

第一步，视觉编码器（通常是 CLIP 的 ViT 或 SigLIP）把一张图切成若干 patch（比如 16×16 像素的小块），每个 patch 变成一个向量。一张 224×224 的图变成 196 个 patch 向量。

第二步，投影层（一个线性层或轻量 MLP）把这 196 个视觉 patch 向量投影到跟 LLM 的 Token Embedding 同一个维度。比如 LLM 的 Embedding 是 4096 维，投影层就把 ViT 输出的视觉向量全映射到 4096 维。

第三步，拼到 LLM 输入里。正常的文本 Token 是 [猫，坐在，垫子上]。加上图片之后变成 [img_patch_1, img_patch_2, ..., img_patch_196, 猫，坐在，垫子上]。LLM 通过自注意力同时看到视觉信息和文本信息，自然能把「图片里那只猫」跟「猫这个字」对齐。

LLaVA 就是这个架构。视觉编码器冻结不动（CLIP 已经训好了），LLM 冻结不动（已有语言能力），只训练中间那层投影层。成本极低，几小时就训完。

VLM 也会幻觉，而且更隐蔽

纯文本幻觉：模型不知道某个事实，编了。VLM 幻觉：模型看错了图片里的东西，然后基于错误的视觉理解做推理。

一张模糊的停车标志图，视觉编码器看成了限速标志。LLM 拿到的是一个「限速标志」的向量，然后一本正经地推理这张图的交通含义。从 LLM 的视角看，它没编造，它收到的视觉输入本身就是错的。

这也是为什么 VLM 对光照、角度、遮挡极其敏感。视觉编码器的感受野有限，远处小物体、被遮挡物体、反光区域都是幻觉重灾区。

面试三问 + 怎么答

「VLM 和纯 LLM 在架构上最大的区别是什么？」— 多了一个视觉编码器 + 投影层。LLM 本身没变。训练时只训投影层（轻量方案）或者把 LLM 也解冻（重量方案）。

「图文怎么对齐的？」— CLIP 的对比学习。核心 loss 让配对的图文向量互信息最大化。对齐质量直接决定 VLM 的上限。

「VLM 的推理成本为什么比 LLM 高？」— 图的 patch 数量远大于文本 Token。224×224 的图 = 196 个 patch = 196 个 Token 的视觉前缀。加上原始文本 Token，序列长度至少翻倍。自注意力的 $O(n^2)$ 随着 Token 翻倍，计算量翻 4 倍。

磨平一些信息差。

第 25 课：大厂真题精讲（上）

字节 · PEFT 方案怎么选

这题考的是决策能力。面试官不只要你背出 LoRA 三个字母，要你展示「我知道什么场景用什么方案，并且知道为什么」。

回答结构分三层。

第一层，先分类。PEFT 有四类：LoRA（低秩分解，加 $B \times A$ 旁路）、QLoRA（LoRA + NF4 量化 + 双重量化，显存压制）、Adapter（在每个子层后插入可训小网络）、Prompt/Prefix Tuning（只训一段可学习的 prompt 向量）。

第二层，给场景。显存充足（>24GB）且追求效果，用 LoRA， $r=8-16$ ， $\alpha=2r$ ，打在 Q 和 V 上效果最好。显存紧张（8-16GB），用 QLoRA，4bit 量化基座，LoRA 参数照常。对精度极度敏感（比如医疗领域），考虑 Adapter，插入位置在 Attention 和 FFN 之后各一层。简单分类或情感分析，Prompt Tuning 就够，只训几十个向量。

第三层，给出你能调的经验值。 r 的选择，简单情感 $r=4$ ，通用任务 $r=8$ ，复杂推理 $r=32$ 。QLoRA 实测显存比理论多 10-20%，因为量化本身有计算开销，而且 Page Attention 的碎片管理也吃显存。

面试官追问：「 $r=8$ 和 $r=32$ 效果差多少？」，通用任务 1-2 个点，复杂推理 3-5 个点。 r 每翻倍收益递减，瓶颈通常在训练数据质量而不是 r 值。别一味堆 r 。

腾讯 · RAG 系统怎么设计

这题考的是工程落地能力。五个关键决策点，每个都有坑。

切分。chunk size 从 300 Token 试起。太小（<100）语义碎片化，太大（>500）检索粒度粗。overlap 设 50-100 Token 保证跨 chunk 信息不丢失。坑：固定 size 切分会切断句子，滑动窗口 + 句边界检测是更好的方案。

Embedding。BGE-large-zh 或 text-embedding-3-large，1536 维。对专业领域（医学、法律、代码），通用 embedding 效果打折，考虑 domain-specific 微调。坑：embedding 模

型和检索用的向量数据库的维度必须一致，改 embedding 模型要重建索引。

检索。向量检索做语义召回 + BM25 做关键词精确匹配，各取 Top-20，合并去重后用 Cross-Encoder 重排序取 Top-5 送入 LLM。坑：纯向量检索同义词不准（「手机壳」和「手机保护套」embedding 距离可能很大），纯关键词检索数字/型号不准（「iPhone 15」和「iPhone 15 Pro」在 BM25 里是不同 token）。

Prompt 设计。每个 chunk 标来源、页码、相似度分数。必须加约束：「如果资料中没有相关信息，直接说不知道，不要编造。」坑：不加约束会导致模型在检索失败时编答案。

评估。RAGAS 框架：检索质量（Precision/Recall）、生成质量（Faithfulness/Relevance）、端到端质量。坑：只测线上指标不看 badcase，上线后用户问「这个政策更新了吗」才发现知识库已经过期。

面试官追问：「chunk size 调到多少合适？」，反问：「你们的文档是什么类型？API 文档 200-300 Token，综述类长文 400-500，FAQ 可以更短 100-150。」

阿里 · 偏差和方差

这题考的是模型诊断能力。核心是读懂训练 Loss 和验证 Loss 的 gap。

偏差高（欠拟合）：训练 Loss 和验证 Loss 都高，gap 小。模型太简单，数据里真正的模式都学不到。解法：加参数、加数据、加训练步数、换更大的基座。

方差高（过拟合）：训练 Loss 低但验证 Loss 高，gap 大。模型记住了训练数据的噪声，泛化能力差。解法：正则化（Dropout、权重衰减）、更多数据、早停。PEFT 天然低方差，因为只训 0.1% 的参数，不太可能过度记忆训练集。全参微调 + 小数据集 = 方差极高，1000 条数据全参微调 7B 模型基本是背答案。

回答公式：「先看两个 Loss 的 gap。gap 小吃两个都高 → 欠拟合。gap 大吃训练低验证高 → 过拟合。PEFT 天然低方差，全参微调数据少时方差极高。」

面试官追问：「你是怎么排查过拟合的？」，「对比训练集和测试集性能。如果训练集 95% 而测试集 70%，八成过拟合。回看数据量，少于 5000 条做全参微调就是在玩火。」

美团 · Agent 怎么拆

这题考的是架构设计。按 ReAct 循环拆四层。

感知层：接收所有输入，用户原始 query、工具返回结果、对话历史、系统状态。一个清晰的观察格式是「工具名称 + 返回数据 + 时间戳」，防止模型把旧数据和当前提问搞混。

决策层：ReAct 模式，Thought → Action → Observation 循环。不是一次性想好所有步骤，而是想一步做一步看反馈。关键：工具调用成功后追加 Observation，调用失败后追加 Error Message 作为新的 Observation，让模型自己决定重试、换参数还是换工具。

执行层：Function Calling。提前定义工具 JSON schema，模型输出调哪个函数传什么参。外部框架执行后将结果注入对话历史。坑：模型可能调一个不存在的函数名，必须在解析层做校验和 fallback。

记忆层：短期记忆 = 对话历史窗口（最近 N 轮），长期记忆 = 向量数据库存关键信息（用户偏好、历史决策）。短期记忆容易被长上下文挤掉，长期记忆需要定期清理和向量检索阈值。

三层防御：参数校验（schema 不匹配直接打回）、异常重试（N 次换参换工具）、回退兜底（全失败输出「我暂时无法完成这个任务，请稍后重试」）。

面试官追问：「多 Agent 和单 Agent 什么时候选多 Agent？」，「任务有清晰子任务边界且可以并行时。比如一个 Agent 拉数据，一个 Agent 做分析，一个 Agent 写报告。注意通信死锁和超时，第一天就要设计好。」

小红书 · MoE 怎么玩更稳

这题考的是前沿架构的理解深度。MoE 的核心挑战有两个，负载均衡和通信开销。

负载均衡：路由网络很容易出现「专家网红」，80% 的 Token 全涌向 2-3 个专家，其余专家闲置。解法是加辅助损失，强制各专家使用率均等。损失公式是各专家平均激活概率的方差，方差越大惩罚越重。调参时辅助损失系数从小到大加，0.01 起步，看实际负载分布调到 0.05-0.1。

通信开销：MoE 的专家分散在多张 GPU 上。每个 Token 的 All-to-All 通信，从路由 GPU 发送到专家 GPU，算完再发回来。Token 量和 GPU 数量线性增长时，通信可能超过计算时间。解法：专家放置到同节点内优先、通信和计算重叠（当前 Token 通信时算下一个 Token）、减少每次通信的 Token 打包量。

训练稳定性：MoE 比 dense 模型更难训。学习率要更保守（同规模 dense 的 0.5-0.7 倍），因为专家网络参数量小更容易震荡。频繁检查负载分布，一旦某个专家连续多步负载

低于阈值（如 0.5%），检查是否「死专家」。

面试官追问：「SwiGLU 和 MoE 是什么关系？」，「SwiGLU 是 FFN 层的激活函数（Swish + 门控线性单元），MoE 是 FFN 层的组织方式（多个专家 + 路由）。DeepSeek-V3 同时用了两者：MoE 组织专家，每个专家内部用 SwiGLU 激活。」

磨平一些信息差。

第 26 课：大厂真题精讲（下） + 系统设计 + 项目讲述

京东 · 模型上线评估闭环

这题考的是工程运维思维。面试官要的不是「上线就完了」，而是「上线后怎么知道好不好 + 不好了怎么修」。

部署层：推理引擎选 vLLM 或 TGI，量化到 INT8 或 FP16，配好 KV Cache 和 PagedAttention 减少显存碎片。单机单卡用 vLLM，多机多卡加 Tensor Parallel。

监控层四个核心指标。延迟 P50/P99，P50 看用户体验，P99 找长尾问题。P99 突然飙升优先排查 KV Cache 超限或单个超长序列拖累 batch。吞吐 (token/s)，吞吐掉可能 GPU 在等 IO 而不是在算，检查 embedding 调用或向量检索是否成了瓶颈。错误率，区分模型超时、工具调用失败、内容安全拦截三类，每类的根因和修复路径不同。显存占用，持续增长没回收代表内存泄漏 (PagedAttention 的 block 没释放)，骤升骤降可能是 batch 大小不稳定。

评估层：离线 benchmark (MMLU、HumanEval 等)，加上业务自建的测试集 (覆盖高频场景、边界 case、长尾问题)。上线前 A/B 测试，新模型 vs 旧模型，看核心业务指标 (而非只看 benchmark 分数)。坑：benchmark 分数涨了但业务指标掉了，因为 benchmark 覆盖不了真实分布。

反馈闭环：badcase 收集 → 分类标注 (幻觉/格式错误/拒绝回答/工具调用失败) → 每月复盘 Top-3 类问题 → 加入训练集 → 月或季度微调更新。

面试官追问：「延迟突然翻了 3 倍你怎么排查？」，「先看 P99 分布，如果集中在少数长序列 → KV Cache 超限导致 swap。再看 GPU 利用率，如果掉下来了 → 等 IO，去查 embedding 服务或向量数据库响应时间。」

百度 · SFT 和对齐原理选路线

这题考的是训练策略决策。核心区分 SFT 和对齐，SFT 教你「说什么」，对齐教你「什么不该说」。

SFT 的阶段目标：让模型学会遵循指令格式，输出有用且有结构的内容。数据是（prompt, 人类写的理想回答）对，Loss 是标准交叉熵。只做 SFT 的模型能回答大部分问题，但不知道拒绝有害请求、不会承认自己不知道、可能输出不安全的建议。

对齐的阶段目标：让模型的输出符合人类偏好，有用、诚实、无害。核心是偏好学习而非模仿学习。数据是偏好对（prompt, good_response, bad_response）而非单条理想回答。

选路线：首期必须 SFT，跳过直接做对齐没有基础对话能力。二期上对齐，选型看预算和安全要求。预算足 + 安全要求高（面向 C 端、涉及敏感话题）→ RLHF，完整 SFT→RM→PPO 流水线，成本最高但效果最稳定。预算紧→DPO，直接用偏好数据一次性梯度更新，不训 RM，计算量约等于一次 SFT。生成任务为主（写作、对话、代码）→GRPO，组内相对评分省掉价值模型。

面试官追问：「你怎么评估 SFT 效果？」，「离线看 benchmark 和人工评估的 win rate。线上看用户留存、回复采纳率、投诉率。SFT 最怕的是看起来变好了但实际上只是背了更多模板。」

滴滴 · Transformer 为什么取代 RNN

表面在问历史，实际考的是对架构缺陷的深层理解。

三个维度答。

并行计算：RNN 必须串行， h_t 依赖 h_{t-1} ， n 个 Token 算 n 步，GPU 永远只用 1 个计算核心等前面的结果。Transformer 的自注意力是一次矩阵乘法 QK^T ， n 个 Token 全部同时算。训练速度差几十到几百倍，对于千亿 Token 级别的预训练，这是能不能训完的差别，不是快慢的问题。

长距离依赖：RNN 靠隐藏状态 h_t 通过 W 反复相乘传递信息，每乘一次信息衰减。LSTM 和 GRU 加了门控缓解但没解决，说到底长距离信息仍然要穿过几十层的递推，每一步都可能丢失。Transformer 的每个词直接通过点积跟所有其他词算相关性。第 1 个词和第 500 个词的「距离」不是 499 步递推，而是一次 QK^T 点积。信息无损。

可堆叠性：没有残差连接的 RNN 堆到 3-4 层就训不动（梯度消失），有残差能到 8-10 层但跟 Transformer 的 100+ 层没法比。深度带来的是抽象层次的提升，浅层学到词间关系，中层学到句法结构，深层学到语义推理。RNN 堆不深，天花板低。

面试官追问：「RNN 还有用武之地吗？」，「有。时间序列预测和流式处理场景 RNN 仍有优势，因为它天然适配时序数据。但 NLP 领域基本被 Transformer 替代了。另外 Mamba 和 RWKV 这些状态空间模型试图在保持线性复杂度的同时达到 Transformer 的效果，是目前最活跃的替代方向。」

开放题 · 挑一个深入研究的模型

这是热情测试。面试官想看你是不是真的对技术有好奇心，还是只会背八股。选你真懂的，别选最火的。

DeepSeek-V3 的回答模板。第一句定性：总参数 1.6T，激活参数 49B，MoE 架构，256 个专家每次激活 Top-8。

三个创新点有序展开。MoE 降低推理成本，256 个专家分散在 GPU 上，每个 Token 只激活 8 个，计算量只有密集模型的 3%，但总容量大了 50 倍以上。负载均衡用辅助损失保证所有专家使用率均等，不出现网红专家。MLA (Multi-head Latent Attention)，KV Cache 的问题在第 21 课讲过，推理时 KV Cache 随序列长度线性增长。MLA 把 K 和 V 投影到比 d_{head} 还小的潜在空间，推理时从潜在空间恢复完整 KV。128K 长上下文的 KV Cache 压到传统方案的 1/10，这是 DeepSeek 支持超长文本又不炸显存的核心原因。GQA，K 和 V 分组共享，Q 独立保留。Llama 2 的 GQA 组数通常是 8，DeepSeek 做进了 MoE 架构里。

面试官追问：「你读过 DeepSeek-V3 的技术报告吗？」，看过就聊，没看过就说「我看了他们的技术博客，核心是 MoE+MLA+GQA 三件套，技术报告太长了还没读完」。坦诚永远比编的好。

系统设计 · 大模型项目架构

七层结构，每层挑一个你真实做过的点展开五句话。别报菜名。

数据层：清洗管道（去重、格式清洗、质量过滤）、标注管道（标注规范、质检机制、标注者间一致性）。你如果搭建过标注平台或者处理过脏数据，重点讲信息清洗的痛点和解决方案。

模型层：选基座（开源 vs 闭源，参数量，许可证）、微调策略（SFT → LoRA/QLoRA → DPO/RLHF）。讲你选基座的逻辑，为什么选 Qwen 不选 Llama，为什么选 7B 不选 13B。

训练层：分布式策略（数据并行 + 模型并行 + ZeRO）、混合精度（FP16/BF16）、梯度检查点。讲你遇到过的最大的训练不稳定问题，OOM、loss 爆炸、收敛慢。

推理引擎层：vLLM/TGI、量化（INT8/INT4）、KV Cache 优化、PagedAttention。讲你做过的一次推理加速，before/after 延迟数字。

应用层：Agent 编排、RAG 管道、Prompt 管理（版本控制、A/B 测试）。讲你设计过的最复杂的 prompt 模板，为什么那么设计。

监控层：延迟、吞吐、错误率、badcase 收集与分类。讲你发现过的线上问题，某个时间段错误率飙升，排查发现是向量数据库挂了。

安全层：输入输出护栏（敏感词过滤、越狱检测）、内容安全模型。讲你对齐过的安全问题，模型曾经输出过什么东西让你觉得必须加护栏。

面试官追问：「你自己搭过哪几层？」，讲你真实搭过的，一层都不搭过就太假了。至少数据层和模型层应该碰过。

讲项目

之前的正反面示范已经很清楚，补充三个面试官做判断的关键点。

第一，模型选型要有理由。不是「我用了 Qwen2.5-7B」，而是「因为业务对中文理解要求高，Llama 中文弱；因为显存只有两张 A100 40GB，70B 跑不动全参，13B 不如 7B + 高质量数据效果好」。

第二，数据量和质量要具体。8000 条客服对话 + 2000 条标注（不要只说「一些数据」）。标注规范是什么，标注者间一致性多少，清洗掉了多少脏数据，这些细节让面试官相信你真的做过。

第三，失败比成功更有说服力。主动讲一个翻车经历：模型上线后用户投诉率涨了 30%，排查发现 SFT 数据里有一条误导性样本被模型过度学习了。回滚、清洗数据、重训、再上线。面试官对「我做的项目全都很顺利拿到很好的结果」的人天然警惕。

Token 是零件。预训练是学拼法。Transformer 是图纸。SFT 和 RLHF 是请师父精进。LoRA 是换零件不换整盒。RAG 是给参考书。Agent 是让乐高自己动起来。

28 课串成了一条线。你能用自己的话讲清楚 Attention 是什么，LoRA 为什么好使，模型为什么必须戴 Mask，这本书就没白写。

磨平一些信息差。

《AI 大模型不难》— 周末读完的 LLM 面试通关书
[GitHub](#)